PROJECT UJIAN TENGAH SEMESTER

PEMROGRAMAN MOBILE

Tsuzuki Aplikasi Pelacak Anime & Komik



Implementasi Git, Dart, Flutter UI, Responsive Layout, Navigation, State Management, Architecture, REST API, Backend, dan Database

Nama	I Kadek Dwi Bajaskara
NIM	2415051068
Kelas	PTI 5A
Tema Project	Media Tracker (Pelacak Anime & Komik)
URL GitHub	https://github.com/shiroKaraa/UTS_2415051068_I_Kadek_Dwi_Bajaskara
Link APK	https://github.com/shiroKaraa/UTS_2415051068_I_Kadek_Dwi_Bajaskara/releases/tag/v1.0.1

Dosen: Dr. Ir. I Ketut Resika Arthana, S.T., M.Kom

Teaching Assistant: Made Yuda Sadewa, S.Pd.

DAFTAR ISI

DAFTAR ISI 2

1. DESKRIPSI APLIKASI 4

2. DAFTAR FITUR APLIKASI 5

2.1. Katalog & Pencarian 5

2.2. Detail & Karakter 5

2.3. Koleksi Pribadi (CRUD) 5

2.4. Pelacakan Progres 5

2.5. Favorit & Label Kustom 6

2.6. Backup & Restore 6

2.7. Multi-Bahasa & Tema 6

2.8. Sensor Konten NSFW 7

2.9. Responsif 7

2.10. Mode Offline & Sinkronisasi 7

2.11. Fitur Tambahan 7

3. ARSITEKTUR APLIKASI 8

3.1. Alur Data 8

3.2. Penjelasan Tiap Layer 9

3.3. Struktur Folder lib/ 10

4. STATE MANAGEMENT 11

4.1. Teknologi 11

4.2. Daftar Provider 11

4.3. Contoh Implementasi 11

4.4. Shared State 13

5. SKEMA DATABASE (SUPABASE / POSTGRESQL) 17

5.1. Tabel 1: koleksi (Tabel Utama) 17

5.2. Tabel 2: label_kustom 18

5.3. Tabel 3: profiles 19

5.4. Operasi CRUD yang Diimplementasikan 19

5.5. RPC Functions (Supabase) 20

6. REST/GRAPHQL API YANG DIGUNAKAN 22

6.1. AniList GraphQL API (Katalog) 22

6.2. Google Translate API (Terjemahan Sinopsis) 24

6.3. GitHub Releases API (Update APK) 24
6.4. Fitur Wajib Terpenuhi 25
7. RESPONSIVE LAYOUT 26
7.1. Breakpointeger yang Digunakan 26
7.2. Contoh Implementasi 26
7.3. Grid yang Berubah Berdasarkan Lebar Layar 27
7.4. Anti Overflow 28
8. DEMO APLIKASI 29
8.1. HALAMAN HOME 29
8.2. HALAMAN KATALOG 31
8.3. HALAMAN DETAIL 33
8.4. FORM TAMBAH KOLEKSI (CREATE) 34
8.5. FORM EDIT KOLEKSI (UPDATE) 37
8.6. HALAMAN KOLEKSI (READ) 38
8.7. HALAMAN LAINNYA (FITUR TAMBAHAN) 40
8.8. HALAMAN PROFIL 42
8.9. RESPONSIVE LAYOUT (COMPACT vs EXPANDED) 43
8.10. BUKTI DATA DARI REST/GRAPHQL API 45
8.11. BUKTI DATA DARI DATABASE (SUPABASE) 48
9. BUKTI PROSES PENGEMBANGAN (GIT LOG) 50
10. KESIMPULAN 54
11. IDENTITAS DAN BUKTI PENGUMPULAN 56

1. DESKRIPSI APLIKASI

Aplikasi: ***Tsuzuki - Media Tracker (Pelacak Anime & Komik)***

Deskripsi:

Tsuzuki adalah aplikasi mobile berbasis Flutter yang dirancang untuk membantu pengguna melacak progres tontonan Anime dan bacaan Komik (Manga, Manhwa, Manhua). Aplikasi ini memanfaatkan AniList GraphQL API sebagai sumber katalog data yang mencakup ratusan ribu judul, serta Supabase (PostgreSQL) sebagai backend database untuk menyimpan koleksi pribadi pengguna secara aman dan tersinkronisasi.

Aplikasi ini menyelesaikan masalah umum yang dihadapi penggemar anime dan komik, seperti lupa episode atau chapter terakhir yang sudah ditonton/dibaca, kesulitan mengelola daftar judul favorit, serta tidak adanya tempat terpusat untuk mencatat rating dan catatan pribadi. Dengan Tsuzuki, pengguna dapat mencari judul, menambahkannya ke koleksi, mencatat progres, memberi rating, dan membuat catatan semuanya dalam satu aplikasi.

Dari sisi teknis, Tsuzuki mengimplementasikan arsitektur berlapis (*layered architecture*) dengan Provider sebagai state management, REST/GraphQL API untuk pengambilan data katalog, serta CRUD database ke Supabase. Aplikasi juga dilengkapi fitur tambahan seperti multi-bahasa (Indonesia & English), sensor konten NSFW, mode offline dengan cache lokal, dan mekanisme update APK otomatis melalui GitHub Releases.

Target Pengguna: Penggemar anime, manga, manhwa, dan manhua yang ingin mengelola koleksi dan progres tontonan/bacaan secara terorganisir.

2. DAFTAR FITUR APLIKASI

Bagian ini merinci seluruh fitur yang tersedia di aplikasi Tsuzuki, dikelompokkan berdasarkan kategori fungsionalnya.

2.1. Katalog & Pencarian

a. Menjelajahi katalog Anime, Manga, Manhwa, dan Manhua dari AniList API.

b. Pencarian judul *real-time* dengan mekanisme debounce (500ms).

c. Filter lanjutan: Genre, Tag, Tahun Rilis, Status Tayang, dan Skor Minimum.

d. Pengurutan: Populer, Skor Tertinggi, Terbaru, A-Z, dan Relevan.

e. Fitur Random Pick untuk merekomendasikan judul secara acak.

2.2. Detail & Karakter

a. Sinopsis lengkap dengan terjemahan otomatis (English → Indonesia).

b. Informasi studio, durasi, total episode/chapter, dan status tayang.

c. Daftar karakter dengan detail voice actor.

d. Judul alternatif (Romaji, English, Native, Sinonim).

e. Preview cover fullscreen dengan zoom & pan.

2.3. Koleksi Pribadi (CRUD)

a. Create: Tambah judul dari katalog ke koleksi pribadi.

b. Read: Lihat koleksi dengan tampilan Grid/List yang dapat dikustomisasi.

c. Update: Edit status, progres, rating, catatan, dan label.

d. Delete: Hapus dengan konfirmasi dialog dan opsi Undo via SnackBar.

e. Aksi Massal: Hapus, ubah status, atau ubah label beberapa item sekaligus.

f. Deteksi Duplikat: Peringatan jika judul sudah ada di koleksi.

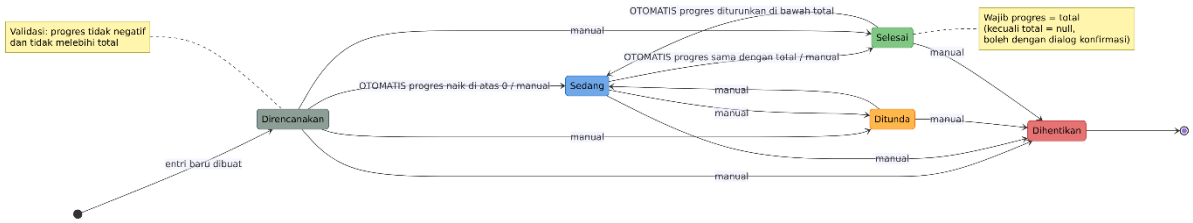
2.4. Pelacakan Progres

a. Tombol cepat +1 / -1 untuk menambah/mengurangi episode/chapter.

b. Auto-update status: Jika progres mencapai total, status otomatis menjadi "Selesai".

c. Validasi progres: Tidak boleh negatif dan tidak boleh melebihi total.

d. Saran otomatis: Jika progres penuh tapi status belum "Selesai".



State Siklus Koleksi

2.5. Favorit & Label Kustom

a. Tandai judul sebagai favorit dengan ikon hati.

b. Halaman Favorit dengan filter (Semua / Anime / Komik).

c. Buat label berwarna untuk mengelompokkan koleksi.

2.6. Backup & Restore

a. Ekspor koleksi ke file JSON (dapat dibagikan ke Google Drive/Files).

b. Impor koleksi dari file JSON dengan mode Gabungkan (Merge) atau Ganti Semua (Replace).

c. Backup otomatis sebelum impor mode Replace untuk keamanan data.

2.7. Multi-Bahasa & Tema

a. Mendukung Bahasa Indonesia dan English (real-time tanpa restart).

b. Tema Dinamis: Mode Gelap, Terang, dan Ikuti Sistem.

c. Kustomisasi tampilan: ukuran card (Kecil/Sedang/Besar), jumlah item per baris (2-6), dan mode Grid/List.

2.8. Sensor Konten NSFW

a. Filter konten 18+ di katalog (default: nonaktif).

b. Sensor cover & judul: blur otomatis dengan tap-to-reveal.

c. 3 Tingkat Sensor: Nonaktif, Ringan, Berat (dengan konfirmasi).

d. Sembunyikan badge 18+ untuk screenshot.

2.9. Responsif

a. Layout menyesuaikan ukuran layar (compact, medium, expanded).

b. Bottom Navigation Bar untuk HP, Navigation Rail untuk Tablet/Desktop.

2.10. Mode Offline & Sinkronisasi

a. Cache lokal koleksi menggunakan SharedPreferences.

b. Deteksi koneksi otomatis dengan banner offline.

c. Auto-retry koneksi setiap 30 detik saat offline.

d. Klaim koleksi tamu: Pindahkan data koleksi guest ke akun saat login.'

2.11. Fitur Tambahan

a. Update Aplikasi: Cek update otomatis via GitHub Releases, download, dan install APK langsung dari aplikasi.

b. Mode Tamu: Jelajahi katalog tanpa login (dengan batasan fitur).

c. Debug Info: Halaman diagnosa untuk melihat status koneksi dan error terakhir.

d. Statistik Profil: Total judul, episode, chapter, volume, dan estimasi waktu menonton.

e. Manajemen Storage: Lihat dan hapus cache gambar, koleksi, dan terjemahan.

f. Onboarding: 3 slide pengenalan aplikasi untuk pengguna baru.

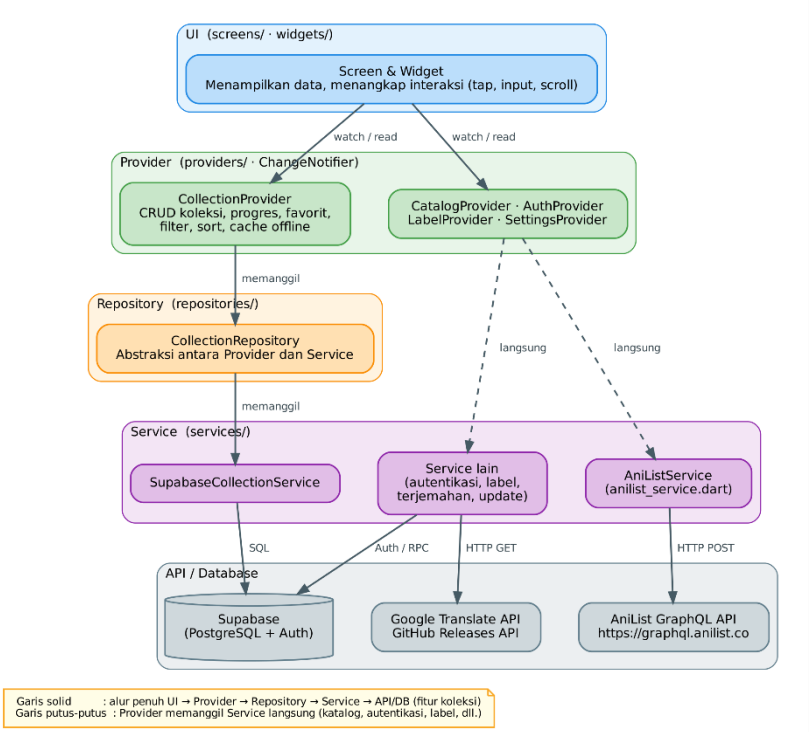
3. ARSITEKTUR APLIKASI

Bagian ini menjelaskan struktur berlapis aplikasi, alur data antar komponen, dan pembagian tanggung jawab tiap layer.

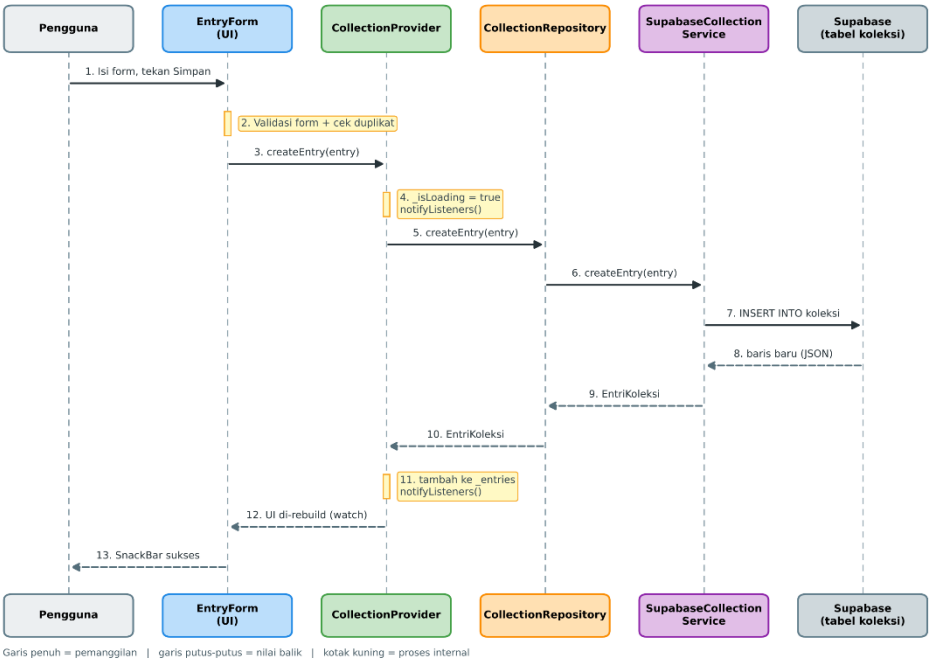
3.1. Alur Data

Aplikasi Tsuzuki menggunakan arsitektur berlapis (layered architecture) untuk memisahkan tanggung jawab antar komponen. Alur penuh UI → Provider → Repository → Service → API/Database diterapkan pada fitur koleksi (CollectionProvider → CollectionRepository → SupabaseCollectionService), yang merupakan integeri CRUD.

Pada provider lain (katalog, autentikasi, label), provider memanggil service secara langsung. Beberapa halaman pendukung (Detail Katalog, Daftar Karakter, Backup/Restore, pengecekan update di Home) memanggil service secara langsung untuk data yang bersifat lokal pada halaman tersebut.



Arsitektur Berlapis



Sequence Alur Data Create

3.2. Penjelasan Tiap Layer

UI (Screen/Widget) Terletak di folder screens/ dan widgets/. Bertugas menampilkan data dan menangkap integereraksi pengguna (tap, input, scroll). UI menggunakan context.watch<Provider>() untuk mendengarkan perubahan state dan context.read<Provider>() untuk mengirim aksi.

Provider (State Management) Terletak di folder providers/. Menggunakan ChangeNotifier untuk mengelola state aplikasi. Provider menyimpan data (loading, data, error) dan memanggil notifyListeners() ketika state berubah, sehingga UI otomatis di-rebuild.Repository Terletak di folder repositories/. Berfungsi sebagai jembatan antara Provider dan Service.

Service (Data Source) Terletak di folder services/. Berisi implementasi konkret untuk berkomunikasi dengan API (AniList GraphQL) atau database (Supabase). Service melakukan HTTP request, JSON parsing, dan error handling.

3.3. Struktur Folder lib/

```
lib/

├── core/      # Konfigurasi inti (tema, warna, ukuran, cache, config)

├── l10n/      # File lokalisasi (terjemahan ID/EN)

├── models/     # Class Dart (POJO / Data Model)

├── providers/   # State Management (ChangeNotifier)

├── repositories/  # Jembatan antara Provider dan Service

├── screens/    # Halaman UI

├── services/    # Logika API & Database

├── widgets/    # Komponen UI Reusable

└── main.dart    # Entry point aplikasi
```

4. STATE MANAGEMENT

Bagian ini menjelaskan penggunaan Provider sebagai solusi *state management*, daftar provider yang ada, dan contoh implementasinya.

4.1. Teknologi

Aplikasi Tsuzuki menggunakan Provider sebagai state management dengan ChangeNotifier sebagai dasar. Pemilihan ini didasarkan pada kesederhanaan, performa, dan integeregrasi yang baik dengan Flutter.

4.2. Daftar Provider

Provider	Fungsi
AuthProvider	Mengelola state autentikasi (login, register, logout, profil pengguna, mode tamu).
CatalogProvider	Mengelola state katalog (data anime/manga, filter, pencarian, pagination, error).
CollectionProvider	Mengelola state koleksi (CRUD, progres, favorit, filter, sort, cache offline).
LabelProvider	Mengelola state label kustom (CRUD label).
SettingsProvider	Mengelola state pengaturan (tema, bahasa, ukuran card, NSFW, cache gambar).

4.3. Contoh Implementasi

Setiap provider mengelola minimal 3 state wajib: loading, data, dan error. Contoh pada CollectionProvider:

```
class CollectionProvider extends ChangeNotifier {

 List<EntriKoleksi> _entries = [];

 bool _isLoading = false;

 String? _errorMessage;

 bool get isLoading => _isLoading;
```

```
String? get errorMessage => _errorMessage;

Future<void> fetchCollection() async {

 _isLoading = true;

 _errorMessage = null;

 notifyListeners(); // UI menampilkan loading

 try {

 _entries = await _repository.getCollection(userId);

 } catch (e) {

 _errorMessage = _friendlyError(e);

 } finally {

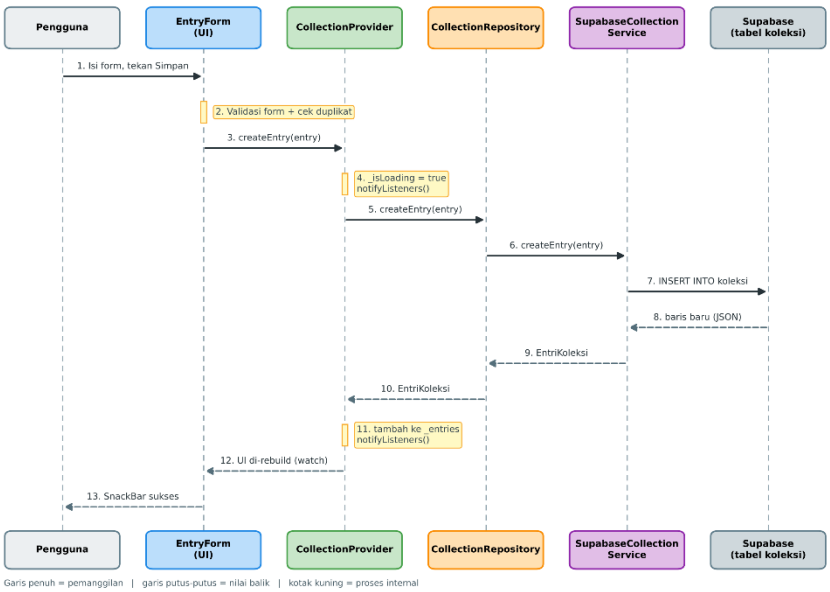
 _isLoading = false;

 notifyListeners(); // UI menampilkan data/error

 }

}

}
```

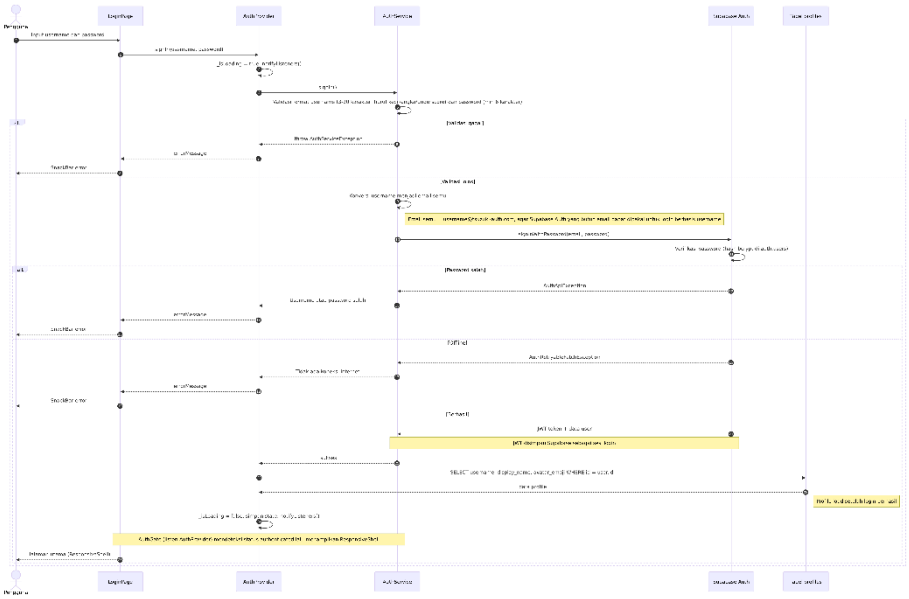


Sequence Create Koleksi

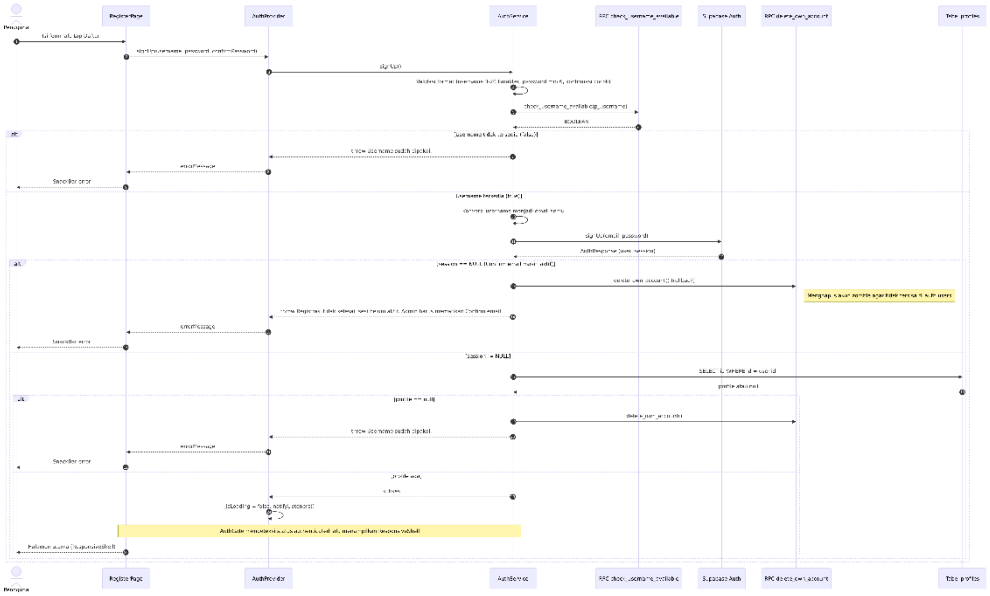
4.4. Shared State

Selain state per provider, aplikasi ini juga mengelola shared state yang dapat diakses lintegeras screen:

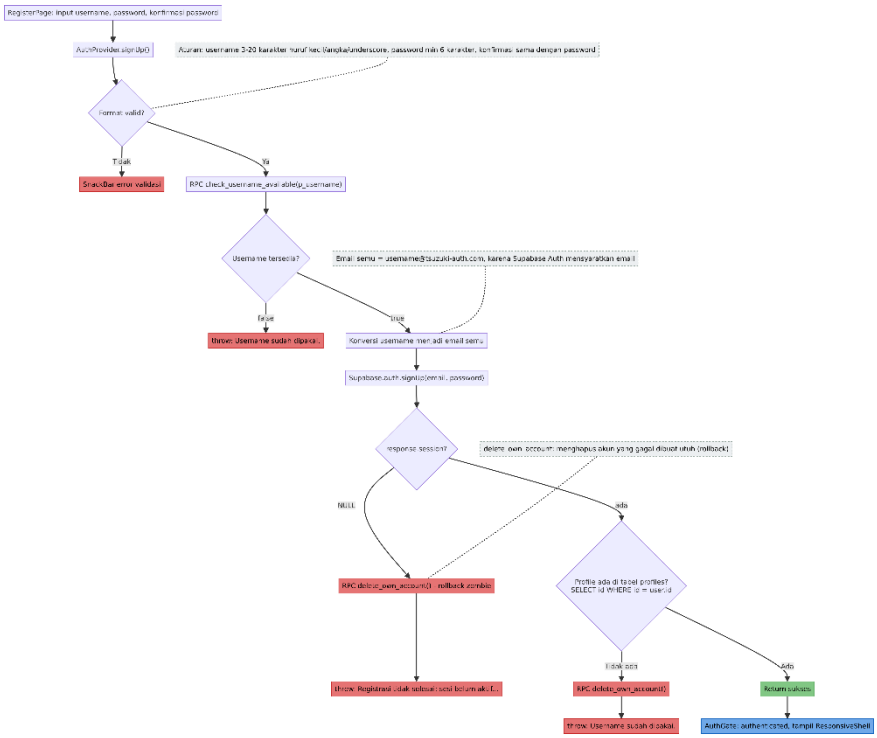
a. Daftar koleksi (CollectionProvider.allEntries) diakses oleh Home, Koleksi, Favorit, Profil.
b. Favorit (CollectionProvider) diakses oleh Home, Koleksi, Favorit.
c. Filter aktif (CatalogProvider) diakses oleh Katalog dan Detail.
d. Tema & Bahasa (SettingsProvider) diakses oleh seluruh aplikasi.



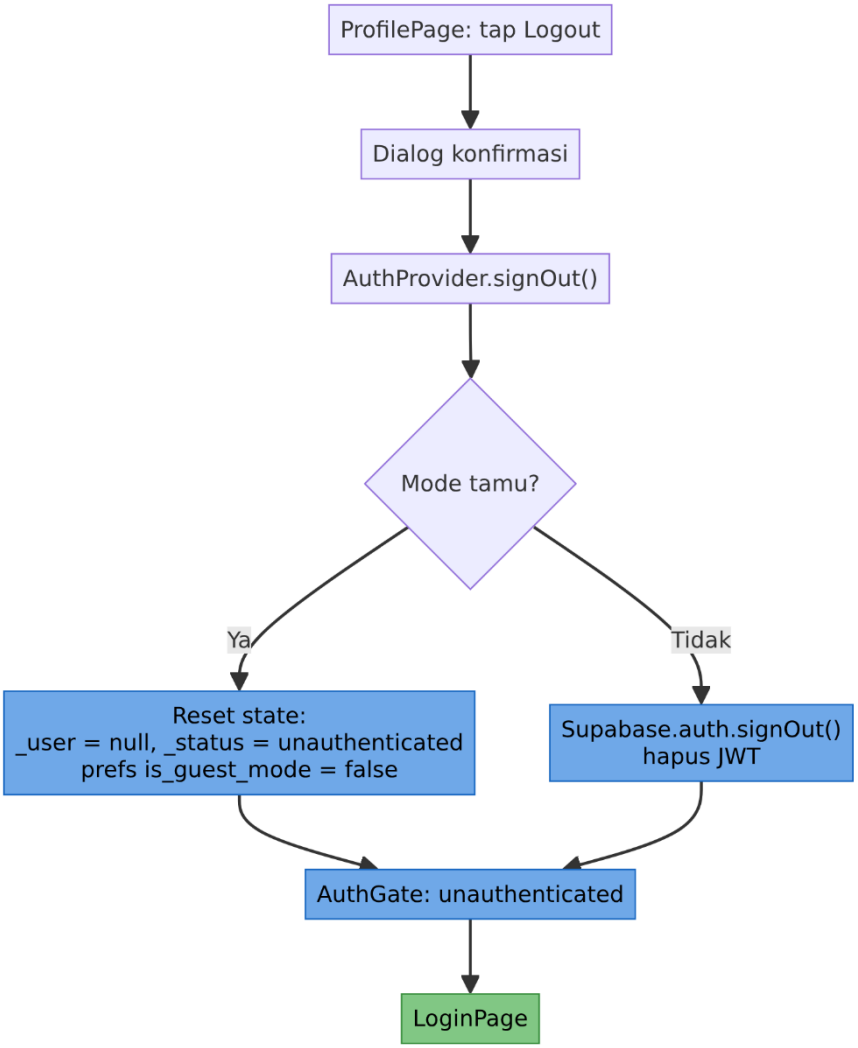
4.1 Sequence Login



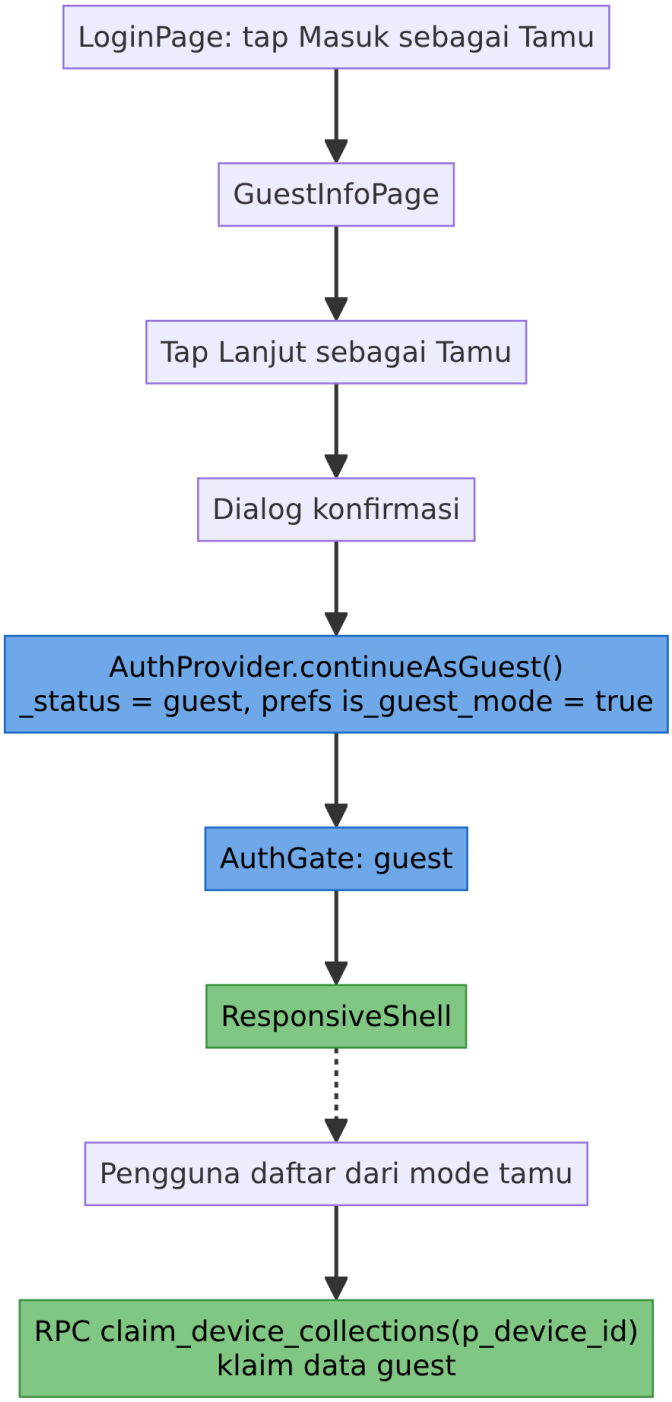
4.3 Sequence Register



4.2a Flowchart Register



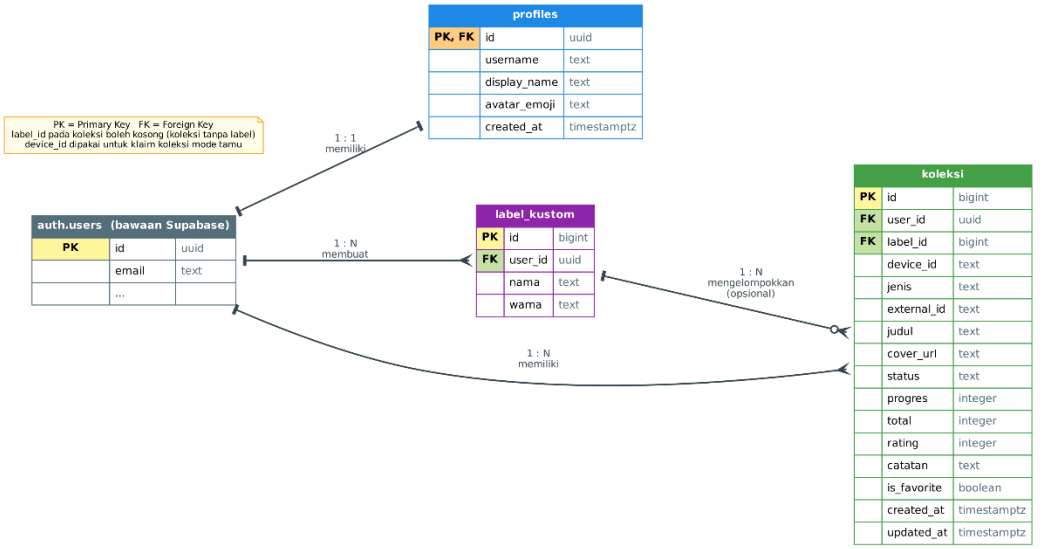
4.2b Flowchart Logout



4.2c Flowchart Mode Tamu

5. SKEMA DATABASE (SUPABASE / POSTGRESQL)

Aplikasi Tsuzuki menggunakan Supabase sebagai backend database. Terdapat 2 tabel data aplikasi (koleksi dan label_kustom) dan 1 tabel profil pengguna (profiles) yang bergantung pada tabel autentikasi bawaan Supabase (auth.users).



ERD Database

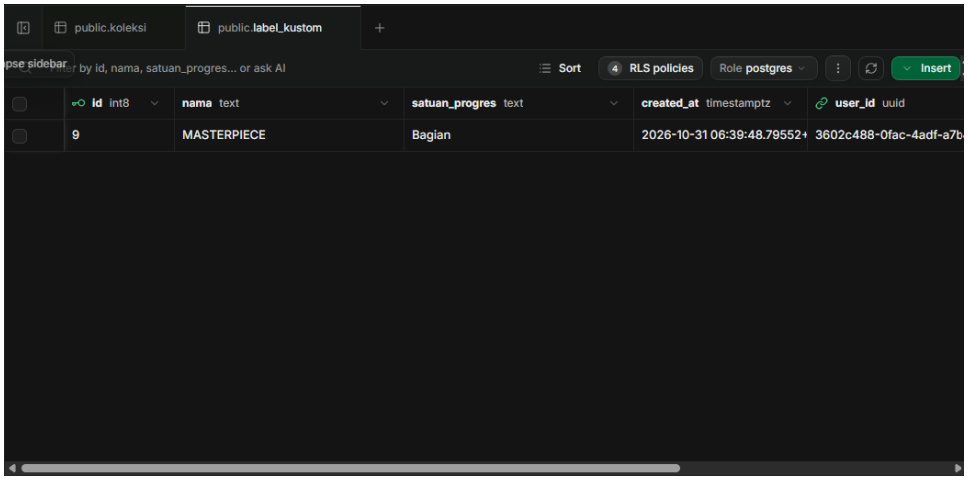
5.1. Tabel 1: koleksi (Tabel Utama)

Kolom	Tipe Data	Keterangan
id	biginteger (PK)	Primary key, auto-increment
user_id	uuid (FK)	Referensi ke auth.users
device_id	text	ID perangkat (untuk klaim data guest)
jenis	text	anime, manga, manhwa, manhua, lainnya
label_id	biginteger (FK)	Referensi ke label_kustom
external_id	text	ID judul dari AniList (contoh: al:12345)
judul	text	Judul anime/komik

cover_url	text	URL gambar cover
status	text	direncanakan, sedang, selesai, ditunda, dihentikan
progres	integer	Episode/chapter yang sudah ditonton/dibaca
total	integer	Total episode/chapter
rating	integer	Rating 1-5
catatan	text	Catatan pribadi
is_favorite	boolean	Status Favorit
created_at	timestamptz	Waktu dibuat
updated_at	timestamptz	Waktu terakhir diubah

Total: 16 kolom (1 primary key + 15 kolom data), memenuhi syarat minimal 4 field + primary key.

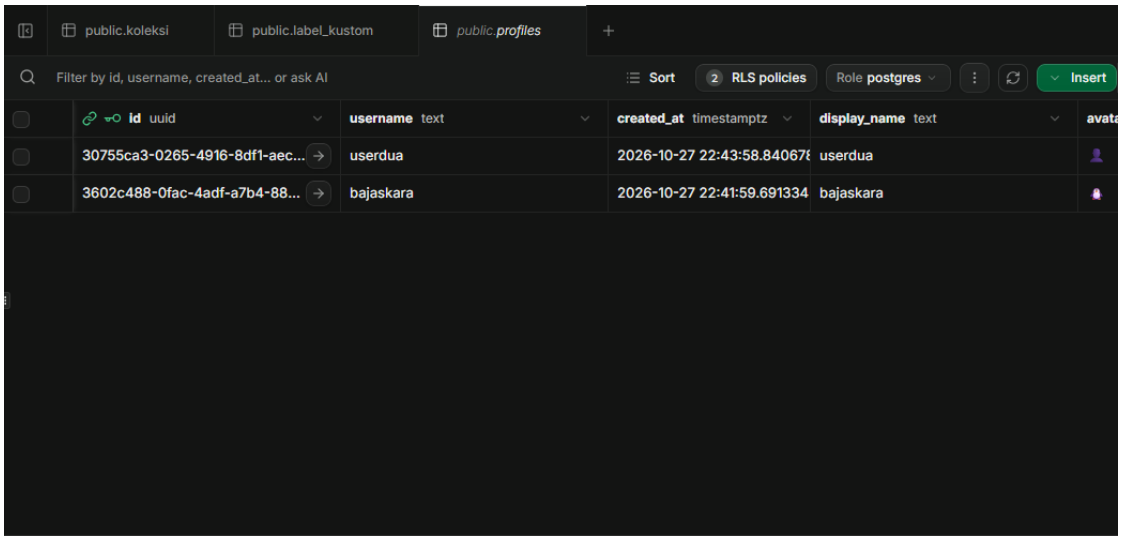
5.2. Tabel 2: label_kustom



Kolom	Tipe data	Keterangan
id	biginteger (PK)	Primary key, auto-increment
user_id	uuid (FK)	Referensi ke auth.users

nama	text	Nama label
warna	text	Kode hex warna (contoh: #5FCB97)

5.3. Tabel 3: profiles



Kolom	Tipe data	Keterangan
id	uuid (PK)	Referensi ke auth.users
username	text	Username unik
display_name	text	Nama tampilan
avatar_emoji	text	Emoji avatar
created_at	timestamptz	Waktu registrasi

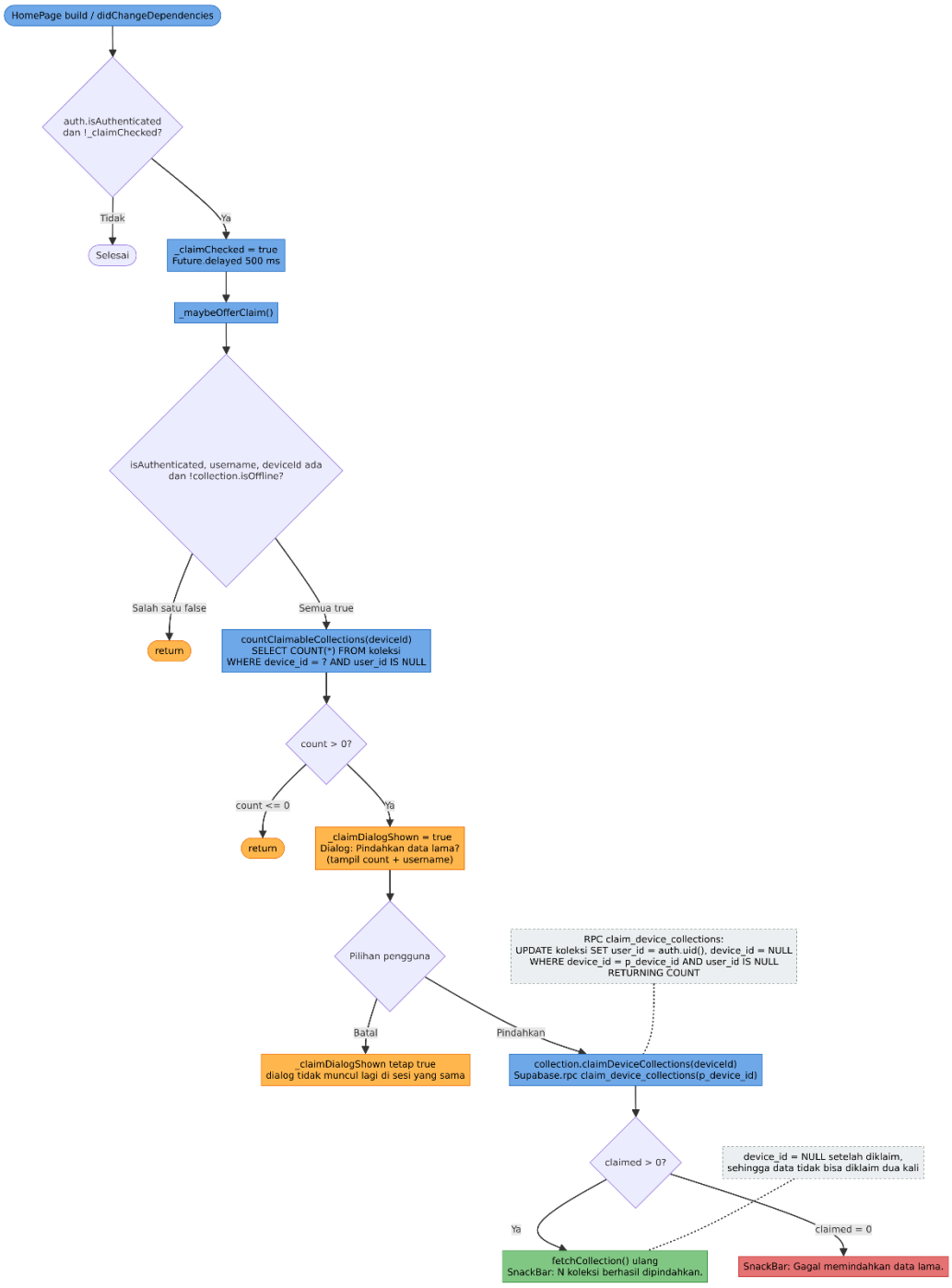
5.4. Operasi CRUD yang Diimplementasikan

Operasi	Method di CollectionProvider	Service
Create	createEntry() / createEntries()	SupabaseCollectionService.createEntry()
Read	fetchCollection()	SupabaseCollectionService.getCollection()

Update	updateEntry() / updateManyStatus()	SupabaseCollectionService.updateEntry()
Delete	deleteEntry() / deleteMany() / deleteAll()	SupabaseCollectionService.deleteEntry()

5.5. RPC Functions (Supabase)

1. check_username_available(p_username) → Cek ketersediaan username.
2. delete_own_account() → Hapus akun sendiri (rollback registrasi).
3. claim_device_collections(p_device_id) → Klaim koleksi dari perangkat guest.



Flowchart Klaim Data Guest

6. REST/GRAPHQL API YANG DIGUNAKAN

Bagian ini merinci API eksternal yang digunakan aplikasi, meliputi endpointeger, metode, fungsi, dan bukti integeregrasinya.

6.1. AniList GraphQL API (Katalog)

- Endpointeger: https://graphql.anilist.co
- Metode: HTTP POST dengan body GraphQL
- Fungsi: Mengambil data katalog anime/manga, detail, karakter, genre, dan tag.
- File Service: *services/anilist_service.dart*

Contoh Query:

```
query ($page: integer, $type: MediaType, $sort: [MediaSort]) {
 Page(page: $page, perPage: 20) {
  pageInfo { hasNextPage total }
  media(type: $type, sort: $sort, isAdult: false) {
   id
   title { romaji english native }
   coverImage { large }
   averageScore
   episodes
   chapters
   status
   genres
  }
 }
}
```

JSON Parsing ke Dart Model (ItemKatalog.fromAniList):

```
factory ItemKatalog.fromAniList(Map<String, dynamic> json, JenisMedia jenis) {
 final titleObj = json['title'] as Map<String, dynamic>?;
 final coverObj = json['coverImage'] as Map<String, dynamic>?;

 double? skor;
 if (json['averageScore'] != null) {
  skor = (json['averageScore'] as num).toDouble() / 10.0;
 }

 return ItemKatalog(
  anilistId: json['id'] as integer,
  judul: titleObj?['romaji'] as String? ?? 'Tanpa Judul',
  judulInggris: titleObj?['english'] as String?,
  coverUrl: coverObj?['large'] as String?,
  skor: skor,
  genres: (json['genres'] as List<dynamic>?)?.cast<String>() ?? [],
  jenis: jenis,
  // ... field lain (total, status, studio, dst.)
 );
}
```

Error Handling:

```
try {
 final response = await _client.post(...).timeout(_timeout);
 if (response.statusCode == 429) throw RateLimitException();
```

```
 if (response.statusCode != 200)

throw ApiException('HTTP ${response.statusCode}');

} on TimeoutException {

 throw NetworkException();

} on http.ClientException {

 throw NetworkException();

}
```

Catatan Operasi GET: Meskipun AniList GraphQL menggunakan HTTP POST untuk query-nya, aplikasi Tsuzuki juga mengimplementasikan dua API dengan metode GET yang datanya ditampilkan di UI:

1. GitHub Releases API (services/update_service.dart)GET https://api.github.com/repos/{owner}/{repo}/releases/latest untuk mengecek update. Data tampil di dialog update.
2. Google Translate API (services/translation_service.dart)GET https://translate.googleapis.com/translate_a/single untuk menerjemahkan sinopsis. Data tampil di halaman detail katalog.

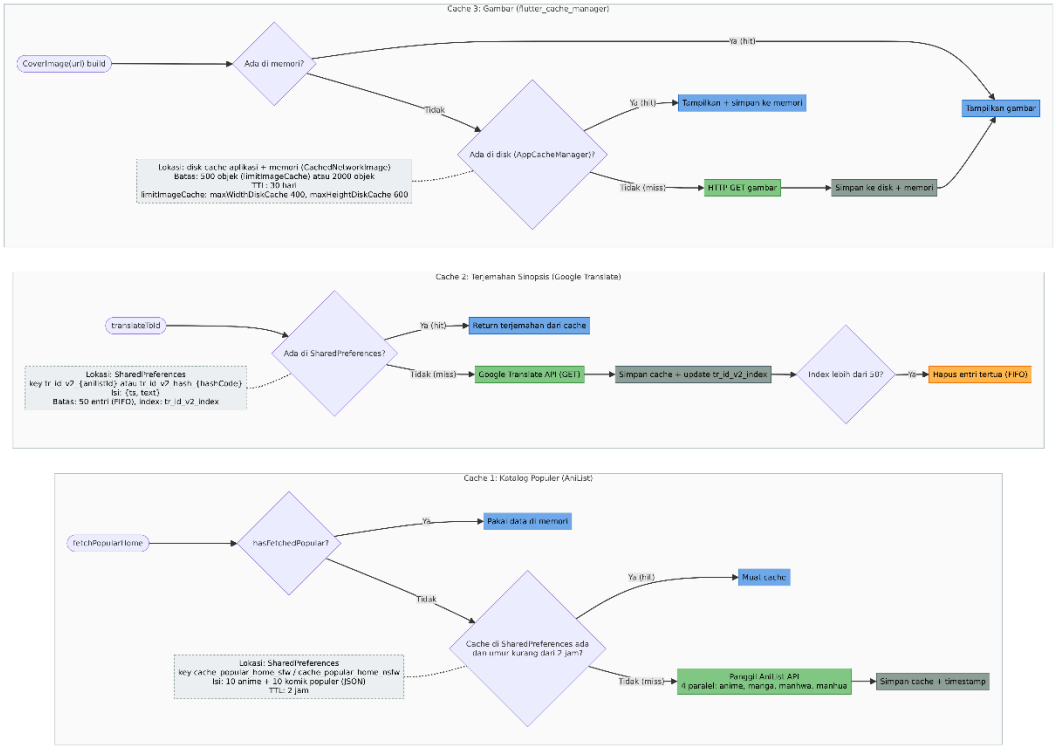
Dengan demikian, ketiga API (AniList, GitHub, Google Translate) memenuhi syarat "menggunakan minimal 1 REST API", "operasi GET", dan "data API ditampilkan di UI".

6.2. Google Translate API (Terjemahan Sinopsis)

- Endpointeger: https://translate.googleapis.com/translate_a/single
- Fungsi: Menerjemahkan sinopsis dari English ke Indonesia secara otomatis.
- File Service: *services/translation_service.dart*
- Caching: Hasil terjemahan disimpan di SharedPreferences (maks. 50 entri).

6.3. GitHub Releases API (Update APK)

- Endpointeger: https://api.github.com/repos/{owner}/{repo}/releases/latest
- Fungsi: Cek versi terbaru aplikasi, download APK, dan install langsung dari aplikasi.
- File Service: *services/update_service.dart*



Caching 3 Lapis

6.4. Fitur Wajib Terpenuhi

Requirement	Status	Bukti
Menggunakan minimal 1 REST API	✅	AniList GraphQL API
Operasi GET wajib	✅	Google Translate API (GET) dan GitHub Releases API (GET) datanya tampil di UI. AniList memakai POST karena GraphQL.
Data API ditampilkan di UI	✅	Halaman Katalog, Detail, Karakter
Response JSON dikonversi ke Dart Model	✅	ItemKatalog.fromAniList(), KarakterAnime.fromEdge()
Loading, error, empty state	✅	SkeletonGridCard, ErrorView, EmptyView

7. RESPONSIVE LAYOUT

Aplikasi Tsuzuki mengimplementasikan responsive layout untuk menyesuaikan tampilan berdasarkan lebar layar perangkat. Implementasi terdapat di core/screen_breakpointegers.dart dan screens/responsive_shell.dart.

7.1. Breakpointeger yang Digunakan

Ukuran	Lebar Layar	Layout
Compact	< 600 dp	Bottom Navigation Bar (5 tab di bawah)
Medium	600 - 840 dp	Navigation Rail (ikon saja, tanpa label)
Expanded	≥ 840 dp	Navigation Rail (ikon + label)

7.2. Contoh Implementasi

```
// lib/core/screen_breakpointegers.dart

class ScreenBreakpointegers {

 static const double compact = 600.0;

 static const double medium = 840.0;



 static bool isCompact(BuildContext context) {

  return MediaQuery.sizeOf(context).width < compact;

 }



 static bool isExpanded(BuildContext context) {

  return MediaQuery.sizeOf(context).width >= medium;

 }

}
```

```
// lib/screens/responsive_shell.dart

LayoutBuilder(

 builder: (context, constraintegers) {

  final width = constraintegers.maxWidth;



  if (width >= ScreenBreakpointegers.medium) {

   return _buildRailLayout(labelType: NavigationRailLabelType.all);

  }



  if (width >= ScreenBreakpointegers.compact) {

   return _buildRailLayout(labelType: NavigationRailLabelType.none);

  }



  return _buildBottomNavLayout();

 },

)
```

7.3. Grid yang Berubah Berdasarkan Lebar Layar

Selain navigasi, grid katalog juga berubah berdasarkan lebar layar dan pengaturan pengguna:

```
final screenWidth = MediaQuery.sizeOf(context).width;

final itemsPerRow = context.watch<SettingsProvider>().itemsPerRow; // 2-6

final columnWidth = (screenWidth - padding - (spacing * (itemsPerRow - 1))) / itemsPerRow;
```

Pengguna dapat mengatur jumlah item per baris (2-6) melalui menu Tampilan.

7.4. Anti Overflow

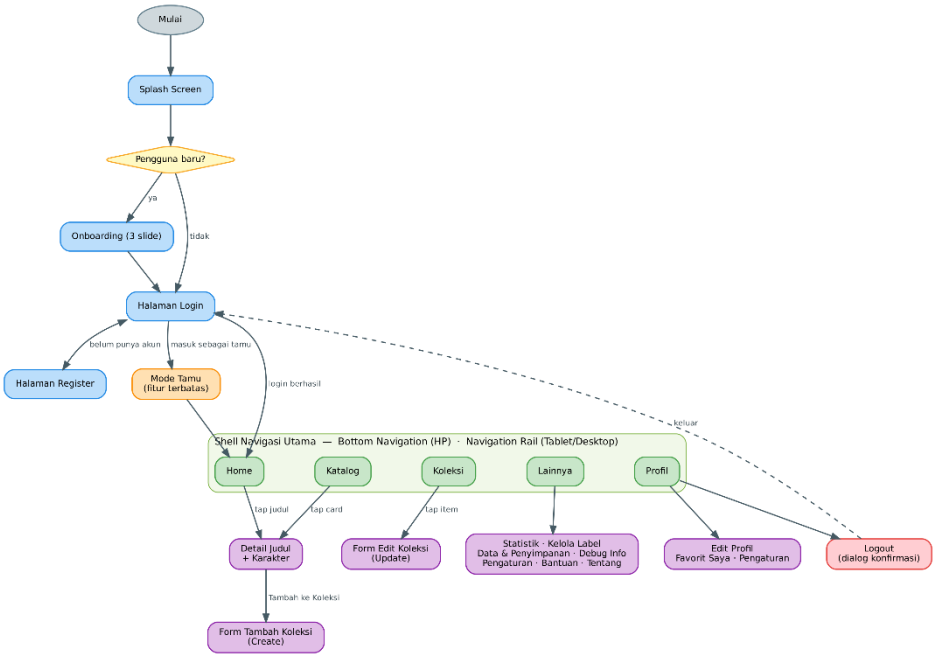
Aplikasi menggunakan beberapa strategi untuk mencegah RenderFlex overflow pada screen utama:

1. Menggunakan Expanded, Flexible, dan ConstrainedBox untuk membatasi ukuran widget.
2. ListView dan GridView untuk konten yang dapat di-scroll.
3. SingleChildScrollView untuk form yang lebih panjang dari layar.
4. FittedBox untuk teks yang berpotensi overflow.

Pengujian manual pada emulator ukuran kecil (360×640) dan tablet (800×1280) menunjukkan tidak ada RenderFlex overflow pada screen utama.

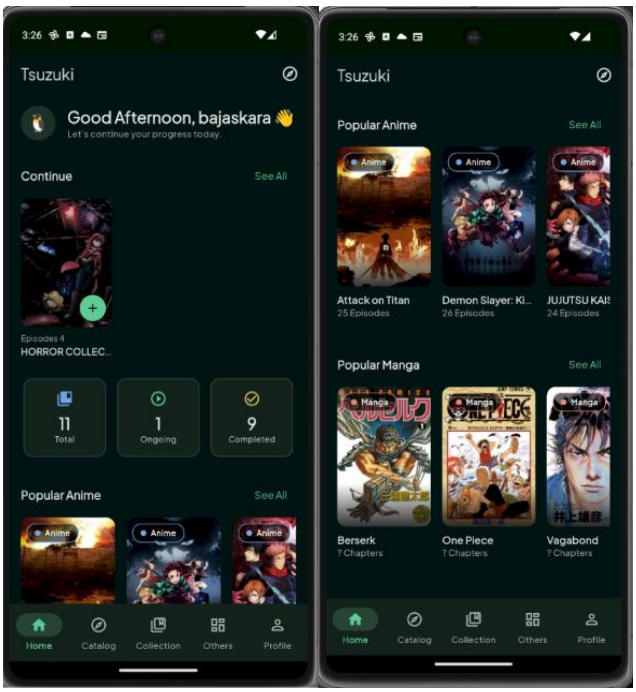
8. DEMO APLIKASI

Bagian ini menampilkan tangkapan layar setiap halaman aplikasi beserta penjelasan fitur yang didemonstrasikan.



Alur navigasi

8.1. HALAMAN HOME



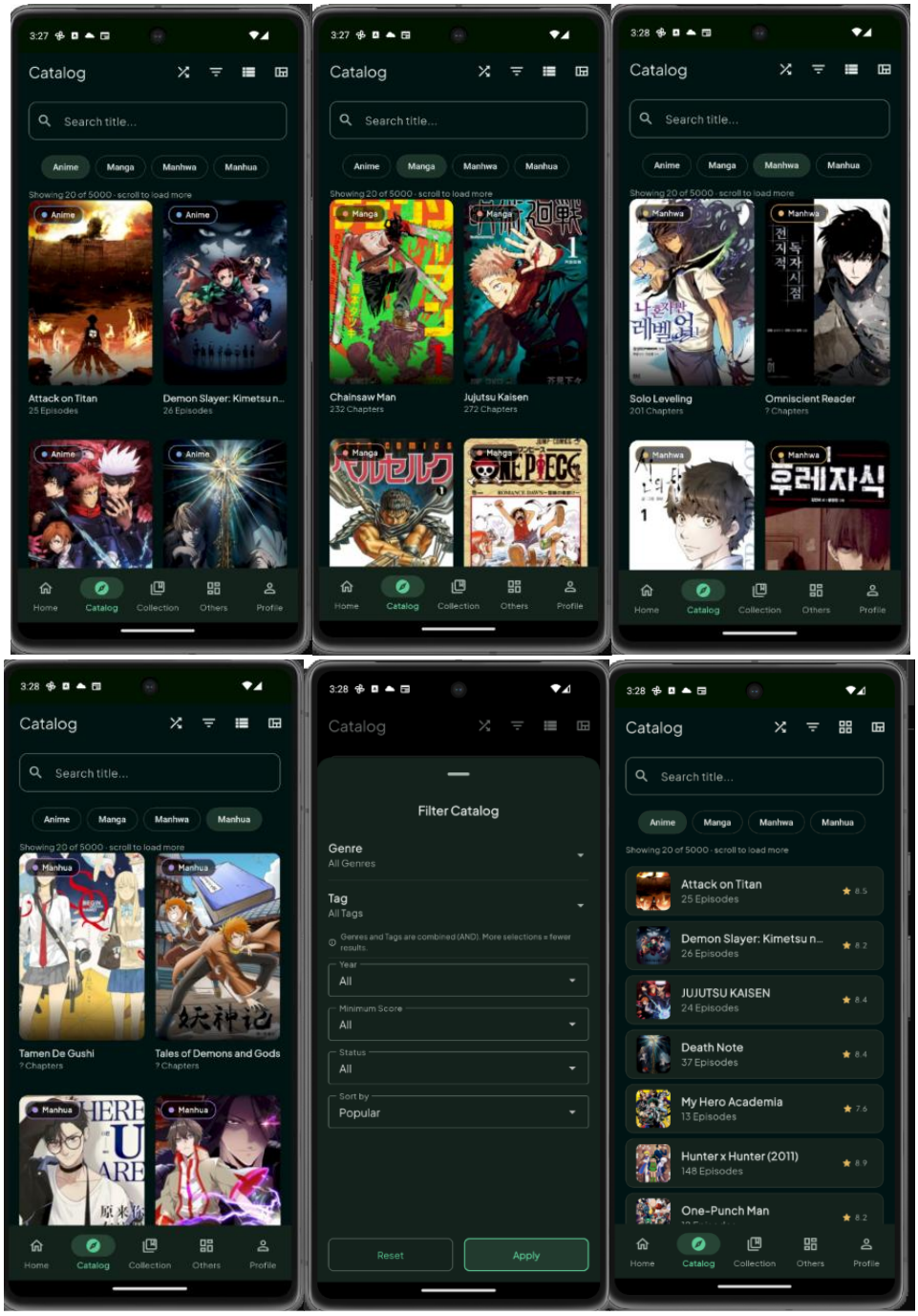
Halaman Home adalah halaman pertama yang dilihat pengguna setelah login. Halaman ini menampilkan:

1. Sapaan Personal "Selamat Pagi/Siang/Malam, [Nama Pengguna] 👋" yang berubah otomatis berdasarkan waktu perangkat.
2. Ringkasan Statistik Koleksi 3 kartu yang menampilkan total judul, judul yang sedang berjalan, dan judul yang sudah selesai. Data ini diambil dari Supabase melalui CollectionProvider.
3. Continue Watching Carousel horizontal yang menampilkan judul dengan status "Sedang" (sedang ditonton/dibaca), dilengkapi tombol +1 untuk menambah progres dengan cepat tanpa membuka detail.
4. Anime Populer Carousel horizontal berisi 10 anime populer dari AniList GraphQL API. Data di-cache selama 2 jam untuk menghemat request.
5. Komik Populer Carousel horizontal berisi 10 komik (Manga/Manhwa/Manhua) populer dari AniList API.
6. anner Offline Jika tidak ada koneksi integerernet, banner kuning muncul di atas menampilkan status offline dengan tombol "Coba Lagi".

Fitur yang Didemonstrasikan:

- ✅ REST/GraphQL API (data anime populer dari AniList)
- ✅ Database (data koleksi dari Supabase)
- ✅ State Management (CollectionProvider, CatalogProvider)
- ✅ Loading state (shimmer saat data belum dimuat)
- ✅ Error state (jika API gagal)
- ✅ Offline mode (banner kuning)

8.2. HALAMAN KATALOG



Penjelasan:

Halaman Katalog adalah pusat pencarian dan penjelajahan judul. Halaman ini menampilkan:

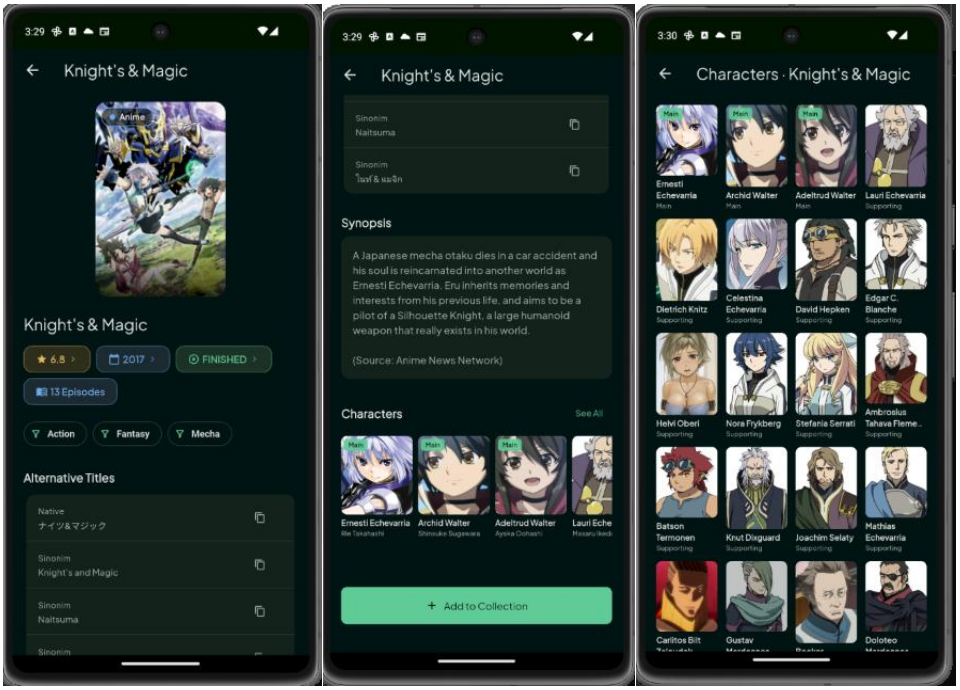
1. Search Bar Pencarian judul real-time dengan debounce 500ms. Data dikirim ke AniList API sebagai parameter search.

2. Tab Jenis Media Chip untuk memilih Anime, Manga, Manhwa, atau Manhua. Setiap tab mengirim query berbeda ke API.
3. Filter Lanjutan Ikon filter membuka bottom sheet dengan opsi: Genre (multi-select), Tag (multi-select), Tahun Rilis, Status Tayang, Skor Minimum, dan Pengurutan.
4. Grid/List View Data ditampilkan dalam bentuk grid poster 2:3 dengan jumlah kolom yang dapat diatur (2-6) atau list view horizontal. Setiap card menampilkan cover, judul, total episode/chapter, badge jenis media, dan badge 18+ (jika konten dewasa).
5. Infinite Scroll Data dimuat bertahap 20 per halaman. Saat pengguna scroll mendekati bawah, halaman berikutnya dimuat otomatis.
6. Random Pick Ikon shuffle di AppBar untuk membuka detail judul acak dari katalog.
7. Loading State Shimmer skeleton grid ditampilkan saat data sedang dimuat.
8. Empty State Tampilan "Tidak Ada Hasil" jika pencarian/filter tidak menemukan judul.

Fitur yang Didemonstrasikan:

- ✅ REST/GraphQL API GET (data dari AniList)
- ✅ JSON Parsing (ItemKatalog.fromAniList)
- ✅ Loading state (shimmer skeleton)
- ✅ Error state (ErrorView dengan tombol retry)
- ✅ Empty state (EmptyView)
- ✅ Filter & search (state management CatalogProvider)
- ✅ Responsive grid (kolom dapat diatur pengguna 2–6 via SettingsProvider.itemsPerRow; di layar lebar pengguna cenderung memilih kolom lebih banyak)

8.3. HALAMAN DETAIL



Penjelasan:

Halaman Detail menampilkan informasi lengkap sebuah judul. Halaman ini didapat dengan passing data dari halaman Katalog (Navigator.push dengan parameter ItemKatalog). Dengan konten yang Ditampilkan:

1. Cover Besar Poster 180x270 dengan Hero animation dari card katalog. Tap untuk melihat preview fullscreen dengan zoom.
2. Judul & Judul Alternatif Judul utama (English/Romaji) dan judul alternatif (Native, Sinonim) dalam section terpisah. Tap ikon copy untuk menyalin.
3. Info Chip Skor (bintegerang), Tahun Rilis, Status Tayang, Total Episode/Chapter. Tap chip untuk membuka katalog dengan filter yang sesuai.
4. Genre Chips Daftar genre. Tap untuk membuka katalog terfilter.
5. Sinopsis Deskripsi lengkap dengan terjemahan otomatis dari English ke Indonesia via Google Translate API. Terjemahan di-cache agar tidak request ulang.
6. Karakter Carousel horizontal karakter (Main, Supporting, Background) dengan foto, nama, dan voice actor. Tap untuk membuka bottom sheet detail karakter.
7. Tombol Aksi "Tambah ke Koleksi" (jika belum ada) atau "Buka Koleksi" (jika sudah ada). Tombol ini melakukan passing data ke form tambah koleksi.

Fitur yang Didemonstrasikan:

✅ Passing data antar-screen (Katalog → Detail)

✅ REST/GraphQL API (detail & karakter dari AniList)

✅ JSON Parsing (ItemKatalog.fromAniList, KarakterAnime.fromEdge)

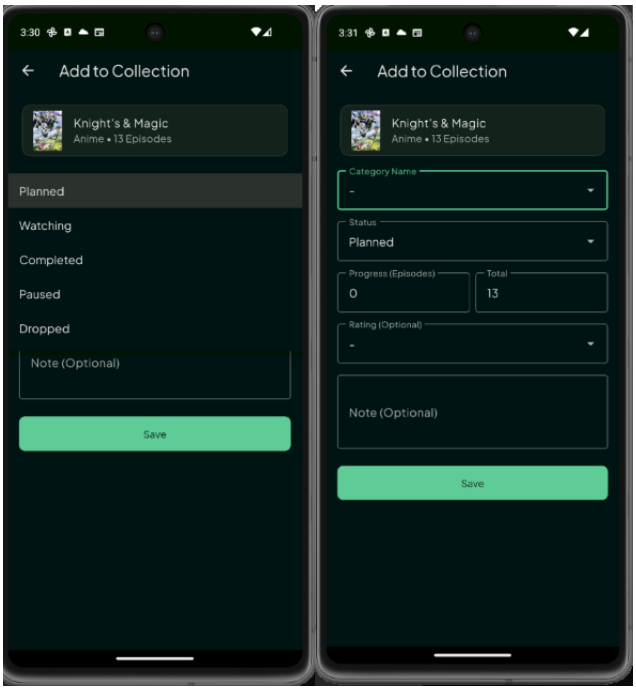
✅ Terjemahan otomatis (Google Translate API)

✅ Loading state (spinner saat memuat detail/karakter)

✅ Error state (jika detail gagal dimuat)

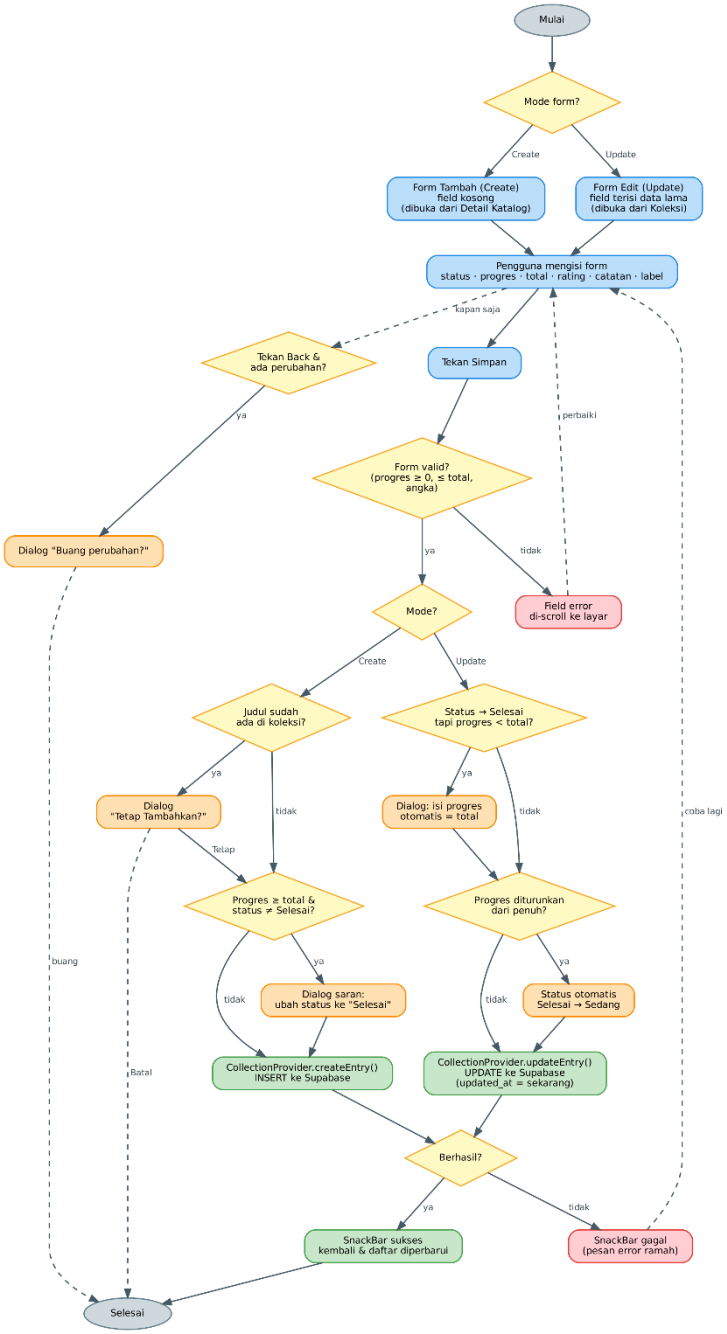
✅ Navigasi (tap genre/tahun → filter katalog)

8.4. FORM TAMBAH KOLEKSI (CREATE)



Penjelasan:

Halaman Create adalah form untuk menambahkan judul dari katalog ke koleksi pribadi. Data yang diisi akan disimpan ke Supabase melalui operasi INSERT.



Flowchart Tambah

Field Form:

a. Label (opsional) Dropdown untuk memilih label kustom (jika pengguna sudah membuat label).

b. Status Dropdown status: Direncanakan, Sedang, Selesai, Ditunda, Dihentikan.

c. Progres Input angka untuk episode/chapter yang sudah ditonton/dibaca. Validasi: wajib diisi, harus angka, tidak boleh negatif, tidak boleh melebihi total.
d. Total Input angka total episode/chapter. Validasi: harus angka positif.
e. Rating (opsional) Dropdown bintegerang 1-5.
f. Catatan (opsional) Textarea untuk catatan pribadi.
g. Tombol Simpan Menyimpan data ke Supabase dan menampilkan SnackBar sukses.

Logika Bisnis:

a. Deteksi Duplikat: Jika judul sudah ada di koleksi (berdasarkan external_id atau judul), muncul dialog konfirmasi "Tetap Tambahkan?".
b. Auto-Suggest Selesai: Jika progres ≥ total tapi status belum "Selesai", muncul dialog saran untuk mengubah status.
c. Validasi Form: Jika form tidak valid, field yang error di-scroll otomatis ke tampilan.
d. Unsaved Changes Warning: Jika pengguna menekan back dengan perubahan belum disimpan, muncul dialog konfirmasi.

Fitur yang Didemonstrasikan:

✅ Form + Validasi (minimal 1 form dengan validasi)

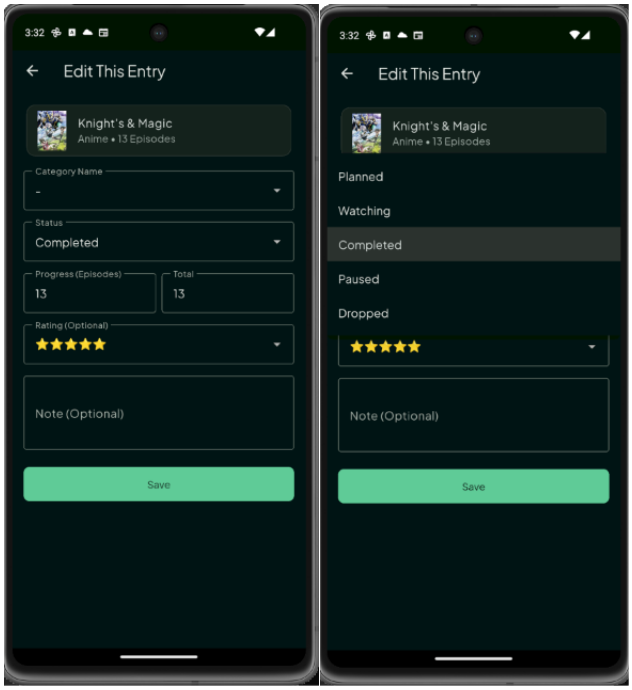
✅ CRUD Database Create (INSERT ke Supabase)

✅ User Feedback (SnackBar sukses/gagal)

✅ Dialog konfirmasi (duplikat, auto-suggest)

✅ State Management (CollectionProvider.createEntry())

8.5. FORM EDIT KOLEKSI (UPDATE)



Penjelasan:

Halaman Edit adalah form untuk mengubah data koleksi yang sudah ada. Halaman ini menggunakan form yang sama dengan Create (EntryFormScaffold), tetapi sudah pre-filled dengan data sebelumnya dan melakukan operasi UPDATE ke Supabase.

Perbedaan dengan Create:

a. Data Pre-filled Semua field (status, progres, total, rating, catatan, label) sudah terisi dengan nilai sebelumnya.
b. Tombol "Simpan Perubahan" Menjalankan operasi UPDATE (bukan INSERT).
c. updated_at otomatis diperbarui ke waktu sekarang.

Logika Bisnis Tambahan:

a. Konfirmasi Status: Jika mengubah status ke "Selesai" tapi progres belum penuh, muncul dialog untuk mengisi otomatis progres ke total.
b. Konfirmasi Turunkan Progres: Jika mengubah progres dari penuh ke kurang, status otomatis berubah dari "Selesai" ke "Sedang".
c. Unsaved Changes: Jika menekan back dengan perubahan belum disimpan, muncul dialog "Buang perubahan?".

Fitur yang Didemonstrasikan:

✅ Form + Validasi

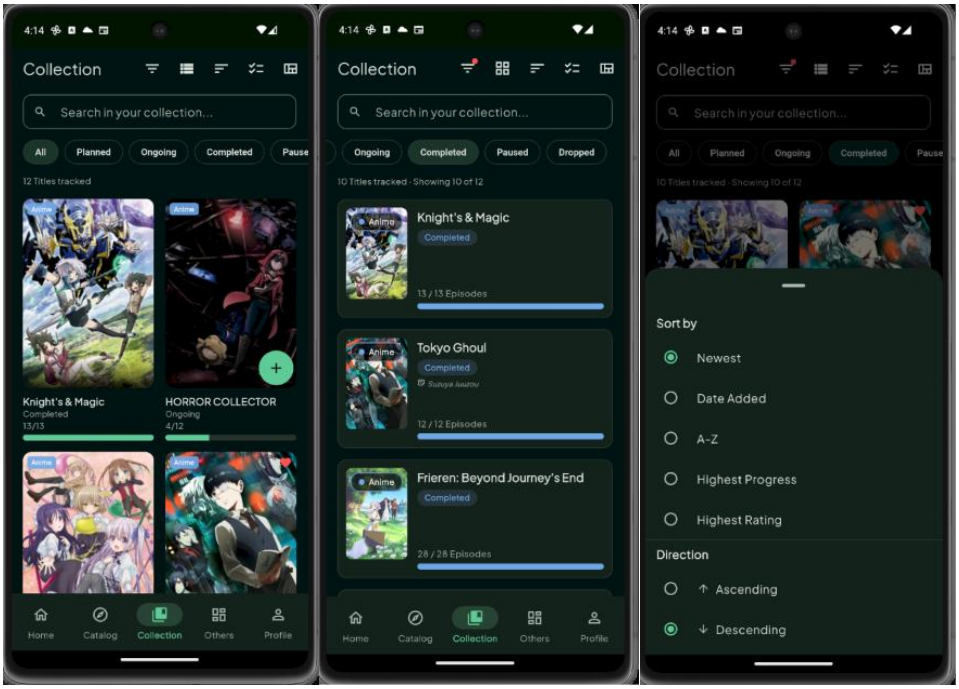
✅ CRUD Database Update (UPDATE ke Supabase)

✅ User Feedback (SnackBar)

✅ Dialog konfirmasi (perubahan status)

✅ State Management (CollectionProvider.updateEntry())

8.6. HALAMAN KOLEKSI (READ)



Penjelasan:

Halaman Koleksi adalah pusat manajemen koleksi pribadi pengguna. Halaman ini merupakan implementasi operasi Read pada CRUD database, mengambil data langsung dari tabel koleksi di Supabase.Dengan konten yang Ditampilkan:

1. Search Bar Pencarian dalam koleksi berdasarkan judul atau catatan (real-time dengan debounce 300ms).
2. Filter Status Chip horizontal: Semua, Direncanakan, Sedang, Selesai, Ditunda, Dihentikan.

3. Grid/List View Kartu koleksi menampilkan cover, judul, jenis media, status (warna berbeda), progres dengan progress bar, label kustom, dan indikator favorit (❤️).
4. Tombol +1 Tombol hijau di setiap card untuk menambah progres episode/chapter dengan cepat tanpa membuka detail.

AppBar Actions:

a. Filter Filter lanjutan (jenis media, rating, status, favorit).
b. Tampilan Toggle Grid/List.
c. Sort Urutkan berdasarkan terbaru, tanggal ditambahkan, judul A-Z, progres, atau rating.
d. Multi-Select Mode pilih untuk aksi massal.
e. Kustomisasi Atur ukuran card dan jumlah item per baris.

Fitur yang Didemonstrasikan:

✅ Database Read operation (SELECT dari Supabase)

✅ State Management (CollectionProvider)

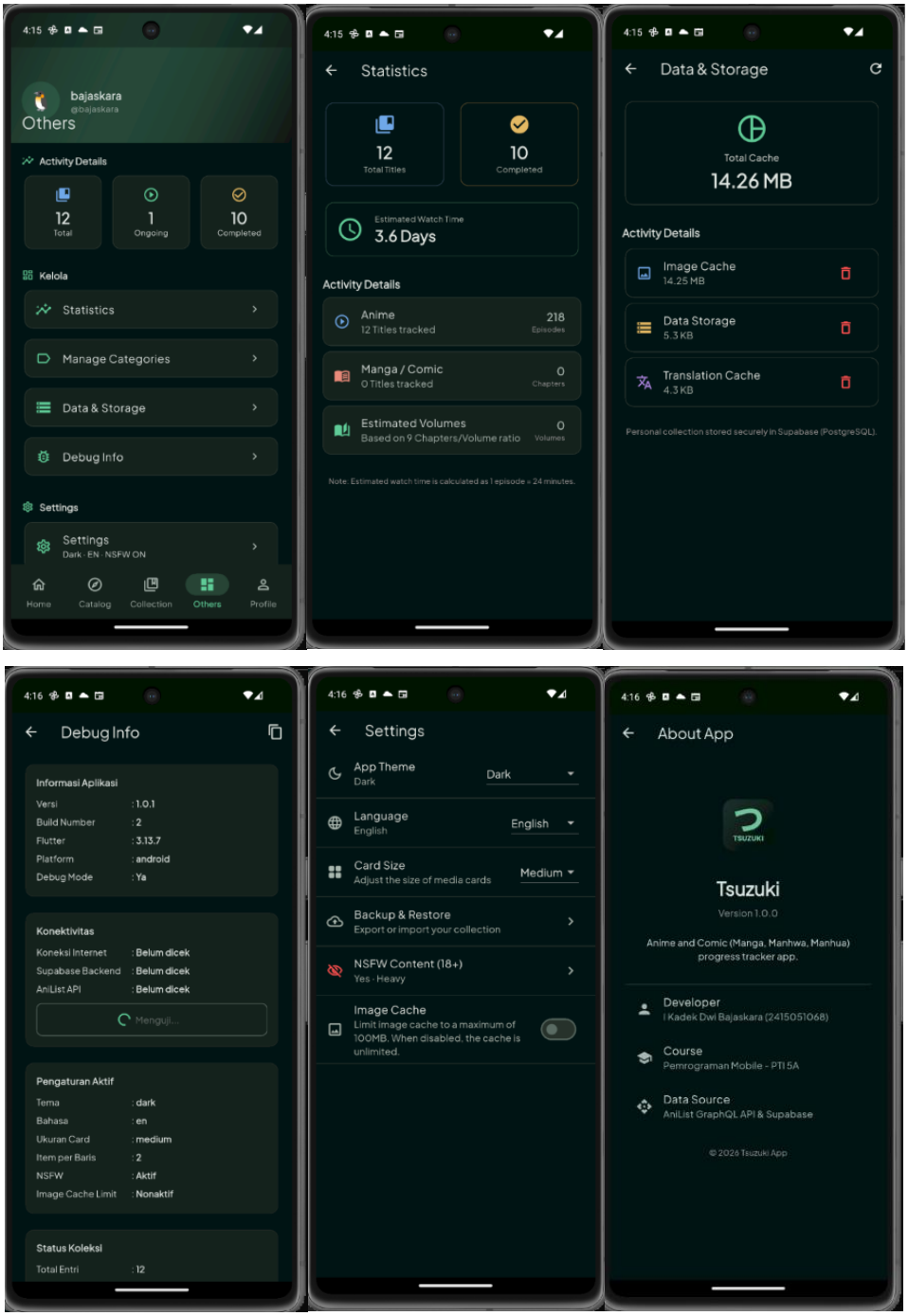
✅ Shared state (favorit & filter aktif)

✅ Aksi massal (hapus, ubah status, ubah label)

✅ Responsive grid

✅ Loading state (shimmer) & Empty state (EmptyView)

8.7. HALAMAN LAINNYA (FITUR TAMBAHAN)



Penjelasan:

Halaman Lainnya adalah pusat navigasi ke seluruh fitur tambahan aplikasi. Halaman ini membuktikan bahwa aplikasi memiliki fitur kreativitas tambahan di luar spesifikasi minimum.Dengan konten yang Ditampilkan:

1. Header Profil Avatar emoji, display name, dan username.

2. Aktivitas Statistik 3 kartu ringkas (Total, Sedang, Selesai) yang dapat diklik untuk membuka koleksi terfilter.

Menu Kelola:

a. Statistik Detail statistik tontonan (total episode, chapter, volume, estimasi waktu).
b. Kelola Label CRUD label kustom berwarna.
c. Data & Penyimpanan Lihat dan hapus cache (gambar, koleksi, terjemahan).
d. Debug Info Halaman diagnosa status koneksi & versi aplikasi.

Menu Pengaturan:

- Pengaturan Tema, bahasa, ukuran card, NSFW, backup/restore.

Menu Bantuan:

a. Dukung Kami, Bantuan & FAQ, Tentang Aplikasi.
b. Footer Logo Tsuzuki dan copyright.

Fitur yang Didemonstrasikan:

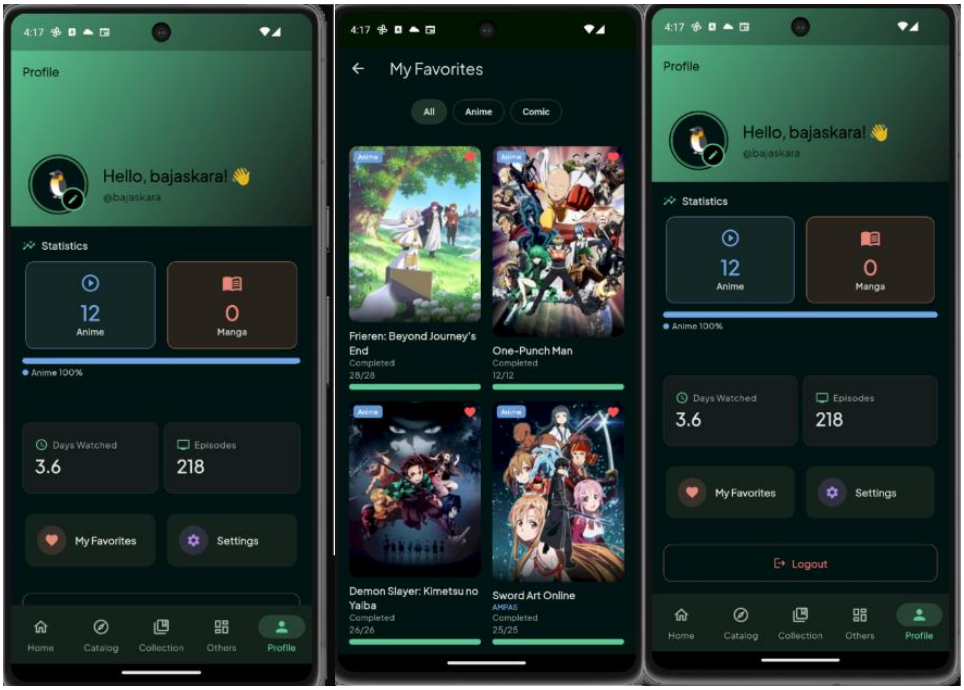
✅ Fitur tambahan di luar spesifikasi (Statistik, Backup, NSFW, Debug, Storage)

✅ Multi-bahasa (ID/EN)

✅ Tema dinamis (Dark/Light)

✅ Navigasi antar fitur

8.8. HALAMAN PROFIL



Penjelasan:

Halaman Profil menampilkan identitas pengguna dan ringkasan aktivitas. Halaman ini menggunakan custom background gradient yang dapat dipilih pengguna di halaman Edit Profil.Dengan konten yang Ditampilkan:

1. Header Profil Avatar emoji besar, sapaan personal ("Halo, [Nama]! 👋"), dan username (@username).

Ringkasan Statistik:

a. Kartu Anime Total judul anime + ikon.
b. Kartu Manga Total judul manga/manhwa/manhua.
c. Ratio Bar Bar horizontal menunjukkan proporsi anime vs manga.
d. Statistik Detail Hari ditonton, episode, chapter, volume.

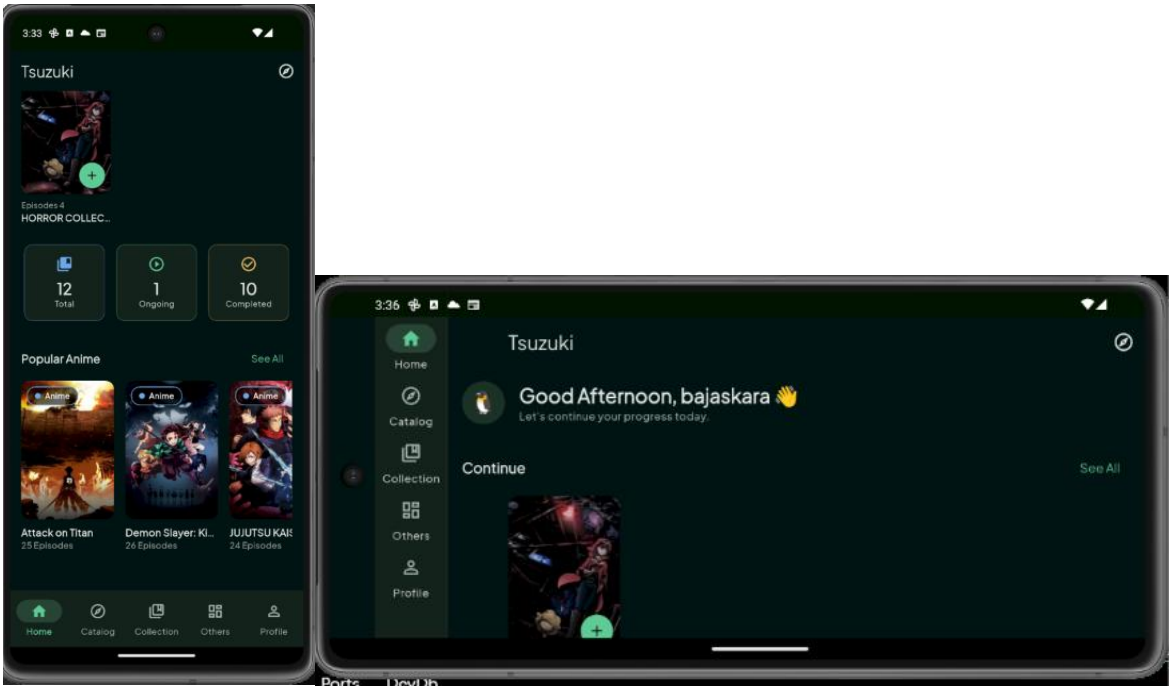
Quick Menu:

a. Favorit Saya Halaman daftar favorit.
b. Pengaturan Halaman pengaturan aplikasi.
c. Tombol Logout Keluar dari akun dengan konfirmasi dialog.

Fitur yang Didemonstrasikan:

- ✅ Autentikasi (AuthProvider)
- ✅ Statistik profil (ProfileStats)
- ✅ Custom background gradient
- ✅ Navigasi ke Favorit & Pengaturan
- ✅ Logout dengan dialog konfirmasi

8.9. RESPONSIVE LAYOUT (COMPACT vs EXPANDED)



Penjelasan:

Halaman ini mendemokan responsive layout dengan 2 kondisi berbeda berdasarkan lebar layar perangkat.

a. Screenshot Kiri Compact (HP, < 600dp)
    1. Navigasi: Bottom Navigation Bar dengan 5 tab (Home, Katalog, Koleksi, Lainnya, Profil) di bagian bawah layar.

2. Jumlah kolom grid ditentukan oleh pengaturan pengguna (SettingsProvider.itemsPerRow, rentang 2–6). Pada layar sempit, pengguna cenderung memilih 2–3 kolom agar poster tetap terbaca.
3. Layout: Konten memenuhi lebar layar, padding kecil.
4. Target Device: Smartphone.

b. Screenshot Kanan Expanded (Tablet/Desktop, ≥ 840dp)

1. Navigasi: Navigation Rail di sisi kiri dengan ikon + label.
2. Pada layar lebar, pengguna cenderung memilih 4–6 kolom agar ruang termanfaatkan. Jumlah kolom tetap mengikuti pengaturan itemsPerRow (2–6), bukan otomatis dari lebar layar.
3. Layout: Konten lebih luas, padding lebih besar.
4. Target Device: Tablet, iPad, atau Flutter Desktop.

Implementasi Teknis

- Responsive layout diimplementasikan menggunakan LayoutBuilder di responsive_shell.dart:

```
LayoutBuilder(

 builder: (context, constraintegers) {

  final width = constraintegers.maxWidth;

  if (width >= 840) return _buildRailLayout(labelType: all);

  if (width >= 600) return _buildRailLayout(labelType: none);

  return _buildBottomNavLayout();

 },

)
```

Grid juga menyesuaikan jumlah kolom berdasarkan itemsPerRow yang dapat diatur pengguna (2-6 kolom).

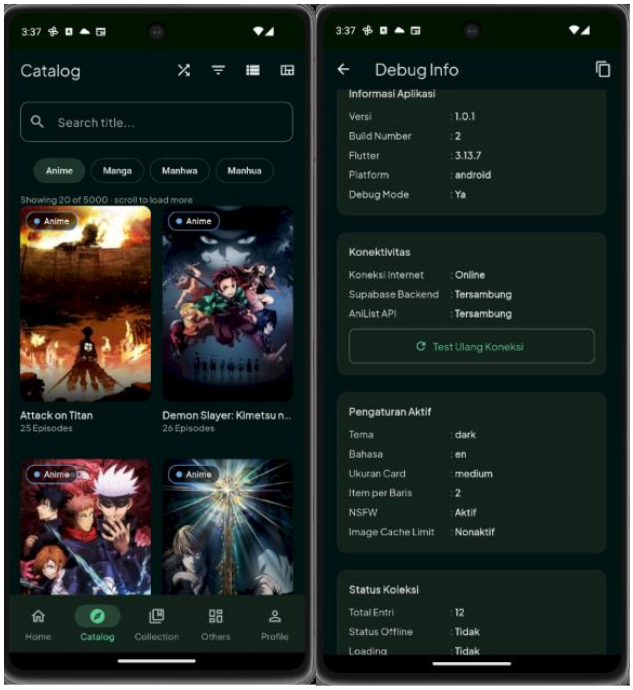
Fitur yang Didemonstrasikan:

✅ Responsive Layout dengan minimal 2 kondisi (compact & expanded)

✅ MediaQuery / LayoutBuilder digunakan

✅ Navigasi berubah otomatis (Bottom Navigation Bar → Navigation Rail). Jumlah kolom grid dapat diatur pengguna (2–6) melalui SettingsProvider.itemsPerRow.

8.10. BUKTI DATA DARI REST/GRAPHQL API



Penjelasan:

Halaman ini membuktikan bahwa aplikasi benar-benar mengambil data dari REST/GraphQL API eksternal (AniList), bukan data hardcoded atau dummy.

1. Bukti 1: Data AniList Tampil di UI
    a. Screenshot di atas menunjukkan halaman Katalog yang menampilkan daftar anime dengan data sebagai berikut:
    b. Judul: "Sousou no Frieren", "One Piece", "Shingeki no Kyojin", dll.
    c. Cover: Gambar poster dari server AniList.
    d. Skor: Rating rata-rata dari AniList (contoh: 9.1, 8.7, 9.0).
    e. Total Episode: Jumlah episode dari AniList.
    f. Status: Sedang Tayang, Tamat, dll.

Semua data ini berasal dari AniList GraphQL API (https://graphql.anilist.co), bukan dari database lokal.

2. Bukti 2: Query GraphQL
   a. Query yang dikirim:

```
query ($page: integer, $type: MediaType, $sort: [MediaSort]) {
 Page(page: $page, perPage: 20) {
  pageInfo { hasNextPage total }
  media(type: $type, sort: $sort, isAdult: false) {
   id
   title { romaji english native }
   coverImage { large }
   averageScore
   episodes
   chapters
   status
   genres
  }
 }
}
```

3. Bukti 3: JSON Parsing

   Response JSON dari AniList di-*parsing* ke Dart model ItemKatalog melalui method ItemKatalog.fromAniList(). Contoh:

```
final json = jsonDecode(response.body);
final mediaList = json['data']['Page']['media'] as List<dynamic>;
```

```
final items = mediaList.map((json) => ItemKatalog.fromAniList(json, jenis)).toList();
```

4. Bukti 4: Operasi GET (Google Translate + GitHub Releases)

   Selain AniList (POST/GraphQL), aplikasi juga memakai dua API dengan metode GET yang datanya tampil di UI:

   a. Google Translate API GET
      https://translate.googleapis.com/translate_a/single. Hasil terjemahan sinopsis tampil di halaman Detail Katalog (screenshot pada halaman 22).
   b. GitHub Releases API GET
      https://api.github.com/repos/{owner}/{repo}/releases/latest. Hasil pengecekan update tampil di dialog "Update Tersedia".

Fitur yang Didemonstrasikan:

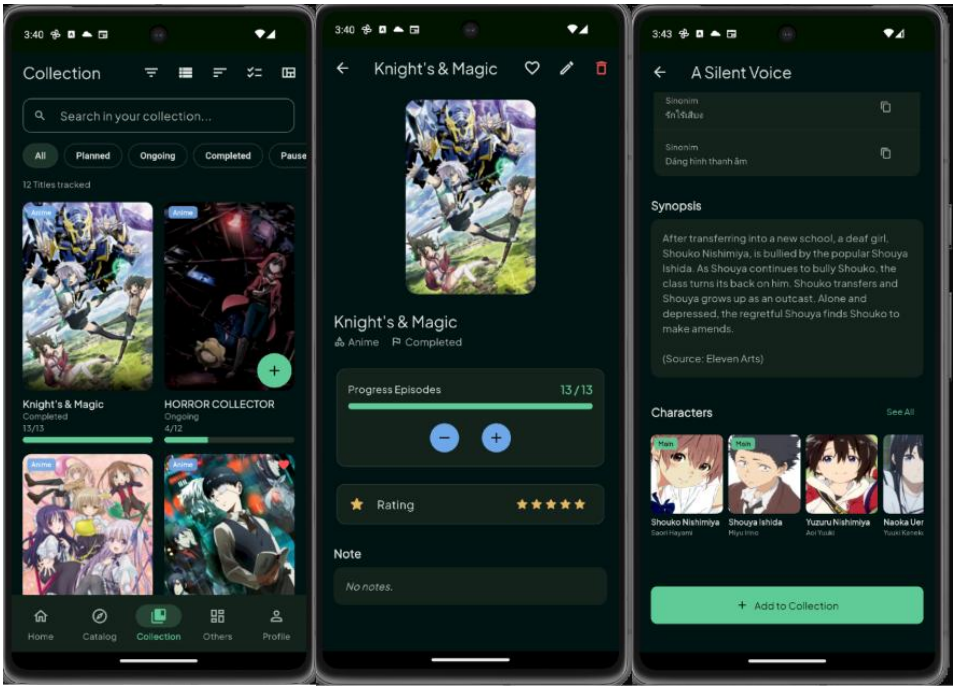
✅ Menggunakan minimal 1 REST API (AniList GraphQL)

✅ Operasi GET: Google Translate API (terjemahan sinopsis tampil di halaman Detail) dan GitHub Releases API (dialog update APK). AniList memakai POST (GraphQL).

✅ Data API ditampilkan di UI (bukan hanya console)

✅ JSON dikonversi ke Dart Model

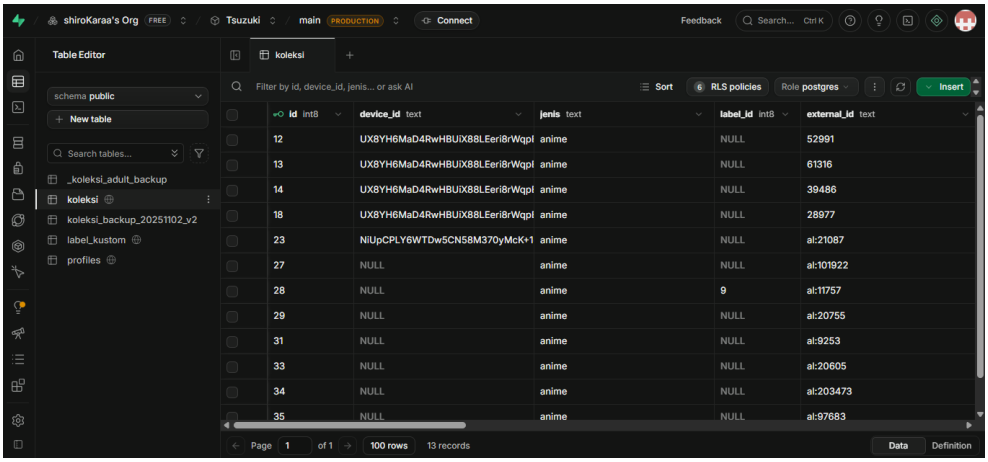
8.11. BUKTI DATA DARI DATABASE (SUPABASE)



Penjelasan:

Halaman ini membuktikan bahwa aplikasi benar-benar menyimpan data ke database Supabase (PostgreSQL), bukan hanya di local storage.

1. Bukti 1: Aplikasi Halaman Koleksi
    a. Screenshot kiri menunjukkan halaman Koleksi di aplikasi yang menampilkan data koleksi pengguna:
    b. Judul anime/komik.
    c. Status (Sedang, Selesai, dll).
    d. Progres episode/chapter.
    e. Label kustom.
    f. Rating & catatan.
2. Bukti 2: Supabase Table Editor



Screenshot kanan menunjukkan Supabase Table Editor yang menampilkan tabel koleksi dengan baris data yang sama dengan yang tampil di aplikasi. Kolom yang terlihat:

id, user_id, jenis, judul, status, progres, total, rating, catatan, is_favorite, created_at.

3. Bukti 3: Operasi CRUD

Aplikasi mengimplementasikan 4 operasi CRUD lengkap:

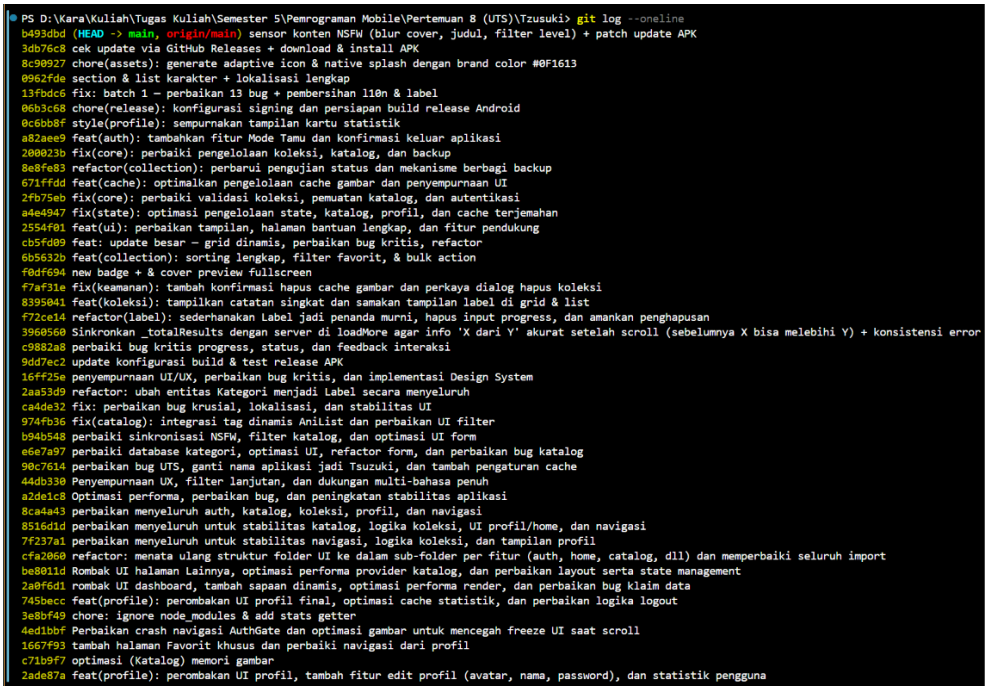
Operasi	Implementasi	Bukti
Create	Tombol "Tambah ke Koleksi" di Detail → INSERT ke Supabase	Data baru muncul di tabel
Read	Halaman Koleksi → SELECT dari Supabase	Data tampil di UI
Update	Form Edit → UPDATE ke Supabase	Data berubah di tabel
Delete	Tombol Hapus + Dialog Konfirmasi → DELETE dari Supabase	Data hilang dari tabel

Fitur yang Didemonstrasikan:

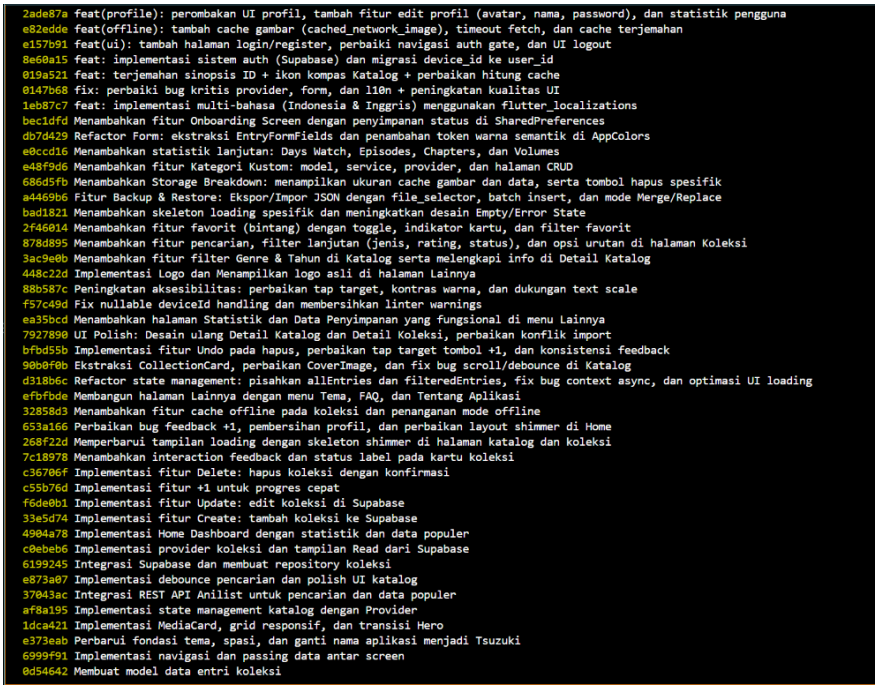
- ✅ Backend database (Supabase / PostgreSQL)
- ✅ Minimal 1 tabel utama dengan primary key + 4+ field
- ✅ CRUD database lengkap (Create, Read, Update, Delete)
- ✅ Delete menggunakan confirmation dialo

9. BUKTI PROSES PENGEMBANGAN (GIT LOG)

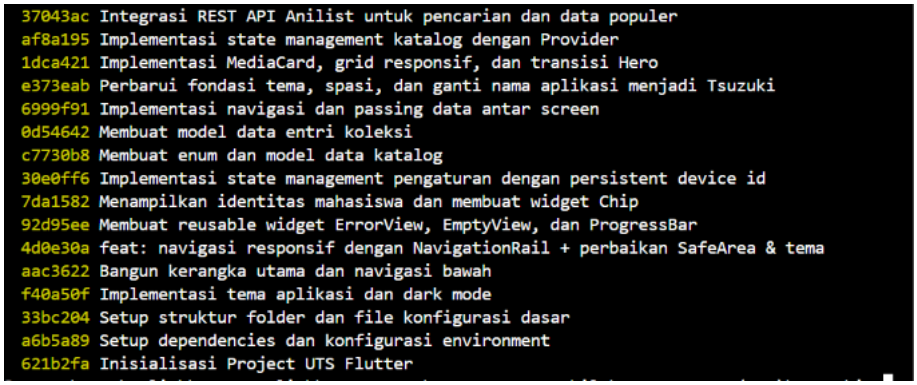
Bagian ini menampilkan riwayat *commit* Git sebagai bukti pengembangan bertahap dari inisialisasi hingga finalisasi.



```
PS D:\Kara\Kuliah\Tugas Kuliah\Semester 5\Pemrograman Mobile\Pertemuan 8 (UTS)\Tzusuki> git log --oneline
b493dbd (HEAD -> main, origin/main) sensor konten NSFW (blur cover, judul, filter level) + patch update APK
3db76c8 cek update via GitHub Releases + download & install APK
8c90927 chore(assets): generate adaptive icon & native splash dengan brand color #0F1613
0962fde section & list karakter + lokalisasi lengkap
13fbdc6 fix: batch 1 — perbaikan 13 bug + pembersihan l10n & label
06b3c68 chore(release): konfigurasi signing dan persiapan build release Android
0c6bb8f style(profile): sempurnakan tampilan kartu statistik
a82aee9 feat(auth): tambahkan fitur Mode Tamu dan konfirmasi keluar aplikasi
200023b fix(core): perbaiki pengelolaan koleksi, katalog, dan backup
8e8fe83 refactor(collection): perbarui pengujian status dan mekanisme berbagi backup
671ffdd feat(cache): optimalkan pengelolaan cache gambar dan penyempurnaan UI
2fb75eb fix(core): perbaiki validasi koleksi, pemuatan katalog, dan autentikasi
a4e4947 fix(state): optimasi pengelolaan state, katalog, profil, dan cache terjemahan
2554f01 feat(ui): perbaikan tampilan, halaman bantuan lengkap, dan fitur pendukung
cb5fd09 feat: update besar — grid dinamis, perbaikan bug kritis, refactor
6b5632b feat(collection): sorting lengkap, filter favorit, & bulk action
f0df694 new badge + & cover preview fullscreen
f7af31e fix(keamanan): tambah konfirmasi hapus cache gambar dan perkaya dialog hapus koleksi
8395041 feat(koleksi): tampilkan catatan singkat dan samakan tampilan label di grid & list
f72ce14 refactor(label): sederhanakan Label jadi penanda murni, hapus input progress, dan amankan penghapusan
3960560 Sinkronkan _totalResults dengan server di loadMore agar info 'X dari Y' akurat setelah scroll (sebelumnya X bisa melebihi Y) + konsistensi error
c9882a8 perbaiki bug kritis progress, status, dan feedback interaksi
9dd7ec2 update konfigurasi build & test release APK
16ff25e penyempurnaan UI/UX, perbaikan bug kritis, dan implementasi Design System
2aa53d9 refactor: ubah entitas Kategori menjadi Label secara menyeluruh
ca4de32 fix: perbaikan bug krusial, lokalisasi, dan stabilitas UI
974fb36 fix(catalog): integrasi tag dinamis AniList dan perbaikan UI filter
b94b548 perbaiki sinkronisasi NSFW, filter katalog, dan optimasi UI form
e6e7a97 perbaiki database kategori, optimasi UI, refactor form, dan perbaikan bug katalog
90c7614 perbaikan bug UTS, ganti nama aplikasi jadi Tsuzuki, dan tambah pengaturan cache
44db330 Penyempurnaan UX, filter lanjutan, dan dukungan multi-bahasa penuh
a2de1c8 Optimasi performa, perbaikan bug, dan peningkatan stabilitas aplikasi
8ca4a43 perbaikan menyeluruh auth, katalog, koleksi, profil, dan navigasi
8516d1d perbaikan menyeluruh untuk stabilitas katalog, logika koleksi, UI profil/home, dan navigasi
7f237a1 perbaikan menyeluruh untuk stabilitas navigasi, logika koleksi, dan tampilan profil
cfa2060 refactor: menata ulang struktur folder UI ke dalam sub-folder per fitur (auth, home, catalog, dll) dan memperbaiki seluruh import
be8011d Rombak UI halaman Lainnya, optimasi performa provider katalog, dan perbaikan layout serta state management
2a0f6d1 rombak UI dashboard, tambah sapaan dinamis, optimasi performa render, dan perbaikan bug klaim data
745becc feat(profile): perombakan UI profil final, optimasi cache statistik, dan perbaikan logika logout
3e8bf49 chore: ignore node_modules & add stats getter
4ed1bbf Perbaikan crash navigasi AuthGate dan optimasi gambar untuk mencegah freeze UI saat scroll
1667f93 tambah halaman Favorit khusus dan perbaiki navigasi dari profil
c71b9f7 optimasi (Katalog) memori gambar
2ade87a feat(profile): perombakan UI profil, tambah fitur edit profil (avatar, nama, password), dan statistik pengguna
```



```
2ade87a feat(profile): perombakan UI profil, tambah fitur edit profil (avatar, nama, password), dan statistik pengguna
e82edde feat(offline): tambah cache gambar (cached_network_image), timeout fetch, dan cache terjemahan
e157b91 feat(ui): tambah halaman login/register, perbaiki navigasi auth gate, dan UI logout
8e60a15 feat: implementasi sistem auth (Supabase) dan migrasi device_id ke user_id
019a521 feat: terjemahan sinopsis ID + ikon kompas Katalog + perbaikan hitung cache
0147b68 fix: perbaiki bug kritis provider, form, dan l10n + peningkatan kualitas UI
1eb87c7 feat: implementasi multi-bahasa (Indonesia & Inggris) menggunakan flutter_localizations
bec1dfd Menambahkan fitur Onboarding Screen dengan penyimpanan status di SharedPreferences
db7d429 Refactor Form: ekstraksi EntryFormFields dan penambahan token warna semantik di AppColors
e0ccd16 Menambahkan statistik lanjutan: Days Watch, Episodes, Chapters, dan Volumes
e48f9d6 Menambahkan fitur Kategori Kustom: model, service, provider, dan halaman CRUD
686d5fb Menambahkan Storage Breakdown: menampilkan ukuran cache gambar dan data, serta tombol hapus spesifik
a4469b6 Fitur Backup & Restore: Ekspor/Impor JSON dengan file_selector, batch insert, dan mode Merge/Replace
bad1821 Menambahkan skeleton loading spesifik dan meningkatkan desain Empty/Error State
2f46014 Menambahkan fitur favorit (bintang) dengan toggle, indikator kartu, dan filter favorit
878d895 Menambahkan fitur pencarian, filter lanjutan (jenis, rating, status), dan opsi urutan di halaman Koleksi
3ac9e0b Menambahkan fitur filter Genre & Tahun di Katalog serta melengkapi info di Detail Katalog
448c22d Implementasi Logo dan Menampilkan logo asli di halaman Lainnya
88b587c Peningkatan aksesibilitas: perbaikan tap target, kontras warna, dan dukungan text scale
f57c49d Fix nullable deviceId handling dan membersihkan linter warnings
ea35bcd Menambahkan halaman Statistik dan Data Penyimpanan yang fungsional di menu Lainnya
7927890 UI Polish: Desain ulang Detail Katalog dan Detail Koleksi, perbaikan konflik import
bfbd55b Implementasi fitur Undo pada hapus, perbaikan tap target tombol +1, dan konsistensi feedback
90b0f0b Ekstraksi CollectionCard, perbaikan CoverImage, dan fix bug scroll/debounce di Katalog
d318b6c Refactor state management: pisahkan allEntries dan filteredEntries, fix bug context async, dan optimasi UI loading
efbfbde Membangun halaman Lainnya dengan menu Tema, FAQ, dan Tentang Aplikasi
32858d3 Menambahkan fitur cache offline pada koleksi dan penanganan mode offline
653a166 Perbaikan bug feedback +1, pembersihan profil, dan perbaikan layout shimmer di Home
268f22d Memperbarui tampilan loading dengan skeleton shimmer di halaman katalog dan koleksi
7c18978 Menambahkan interaction feedback dan status label pada kartu koleksi
c36706f Implementasi fitur Delete: hapus koleksi dengan konfirmasi
c55b76d Implementasi fitur +1 untuk progres cepat
f6de0b1 Implementasi fitur Update: edit koleksi di Supabase
33e5d74 Implementasi fitur Create: tambah koleksi ke Supabase
4904a78 Implementasi Home Dashboard dengan statistik dan data populer
c0ebeb6 Implementasi provider koleksi dan tampilan Read dari Supabase
6199245 Integrasi Supabase dan membuat repository koleksi
e873a07 Implementasi debounce pencarian dan polish UI katalog
37043ac Integrasi REST API Anilist untuk pencarian dan data populer
af8a195 Implementasi state management katalog dengan Provider
1dca421 Implementasi MediaCard, grid responsif, dan transisi Hero
e373eab Perbarui fondasi tema, spasi, dan ganti nama aplikasi menjadi Tsuzuki
6999f91 Implementasi navigasi dan passing data antar screen
0d54642 Membuat model data entri koleksi
```



Tahap	Milestone	Commit
1	Project Initialization	621b2fa
2	Architecture	33bc284
3	Basic UI	aac3622
4	Responsive Layout	4d0e30a
5	Navigation	6999f91
6	Models	c7730b8, 0d54642
7	State Management	af8a195
8	REST API	37043ac
9	Database	6199245

10	Create	33e5d74
11	Update	f6de0b1
12	Delete	c36706f
13	integereraction	7c18978
14	Error Handling	268f22d, bad1821
15	UI Refinement	7927890, 16ff25e
16	Final	81398be

Penjelasan:

Repository ini memiliki 98 commit bermakna yang mencerminkan proses pengembangan bertahap dari inisialisasi hingga finalisasi. Setiap commit mewakili milestone tertentu, mulai dari setup project, pembuatan UI, implementasi state management, integeregrasi API, hingga CRUD database.

Proses commit yang bertahap ini membuktikan bahwa aplikasi dikembangkan secara iteratif dan inkremental, bukan hanya sekadar "initial commit" dan "final commit".

✅ 98 commit bermakna

- ✅ Push per milestone
- ✅ Riwayat pengembangan yang jelas

10.KESIMPULAN

Aplikasi Tsuzuki berhasil dibangun sebagai aplikasi mobile Flutter yang mengintegeregrasikan seluruh materi dari Worksheet 1-7, meliputi:

1. Dart Fundamentals Penggunaan class, model, collection, null safety, Future, async/await, dan exception handling.
2. Flutter UI Fundamentals Widget tree, layout (Row, Column, Expanded, Stack), list/grid, reusable widget, dan styling.
3. Responsive Layout LayoutBuilder dan MediaQuery untuk menyesuaikan tampilan berdasarkan lebar layar (compact, medium, expanded).
4. Navigation & User integereraction Multi-screen navigation, passing data antar-screen, form dengan validasi, dan user feedback (SnackBar, Dialog).
5. State Management & Architecture Provider + ChangeNotifier dengan arsitektur berlapis (UI → Provider → Repository → Service → API/DB).
6. REST API & Database integeregration AniList GraphQL API untuk katalog, Supabase PostgreSQL untuk koleksi pribadi, dengan operasi GET, JSON parsing, dan CRUD lengkap.
7. Git & GitHub 98 commit bermakna yang mencerminkan proses pengembangan bertahap.

Cakupan Requirement Tambahan:

1. Dart Model (7 class, 4 enum):
   - Class: ItemKatalog, KarakterAnime, EntriKoleksi, LabelKustom, ProfileStats, NsfwSettings, UpdateInfo
   - Enum: JenisMedia, StatusKoleksi, CardSize, NsfwFilterLevel

2. Reusable Widget (20 file di lib/widgets/):

   Antara lain MediaCard, CollectionCard, CollectionListCard, CoverImage, ErrorView, EmptyView, MediaTypeChip, NsfwBlurWrapper, AdultBadge, ProgressBar, ShimmerLoading, SkeletonGridCard, SkeletonListCard, EntryFormFields, AlternativeTitlesSection, CharacterSection, DisplayModeSheet, CoverPreviewDialog, UpdateDialog.

3. Bentuk User integereraction:
    - Tombol cepat +1/−1 progres
    - Toggle favorit (ikon hati)
    - Multi-select aksi massal (hapus/ubah status/ubah label)
    - Undo hapus via SnackBar
    - Filter bottom sheet
    - Preview cover dengan zoom/pan
    - Pencarian dengan debounce (500 ms katalog, 300 ms koleksi)

4. Fitur tambahan yang diimplementasikan sebagai nilai plus:
    - Multi-bahasa (Indonesia & English).
    - Sensor konten NSFW dengan 3 tingkat.
    - Mode offline dengan cache lokal dan auto-retry.
    - Update APK otomatis via GitHub Releases.
    - Backup & Restore koleksi dalam format JSON.
    - Statistik profil dan manajemen storage.

Dengan demikian, aplikasi Tsuzuki memenuhi seluruh spesifikasi minimum yang ditetapkan dalam Project UTS Pemrograman Mobile dan menunjukkan proses pengembangan yang nyata melalui Git/GitHub.

11.IDENTITAS DAN BUKTI PENGUMPULAN

Bagian ini memuat data identitas mahasiswa dan tautan bukti pengumpulan proyek.

Nama	I Kadek Dwi Bajaskara
NIM	2415051068
Kelas	PTI 5A
Mata Kuliah	Pemrograman Mobile
Nama Project	Tsuzuki
Tema	Media Tracker (Pelacak Anime & Komik)
URL GitHub Repo	https://github.com/shiroKaraa/UTS_2415051068_I_Kadek_Dwi_Bajaskara
Nama Branch Utama	main
REST API yang digunakan	AniList GraphQL API (https://graphql.anilist.co); GitHub Releases API (GET) dan Google Translate (GET) sebagai API pendukung
Backend/Database yang digunakan	Supabase (PostgreSQL)
Jumlah Screen	25 screen fungsional (di luar splash)
Jumlah Meaningful Commit	98 commit (tersebar 25 Okt – 7 Nov 2026)
Hash Commit Final	81398be
Link APK / File APK	https://github.com/shiroKaraa/UTS_2415051068_I_Kadek_Dwi_Bajaskara/releases/tag/v1.0.1

README.md	https://github.com/shiroKaraa/UTS_2415051068_I_Kadek_Dwi_Bajaskara/blob/main/README.md
Dokumentasi PDF	https://github.com/shiroKaraa/UTS_2415051068_I_Kadek_Dwi_Bajaskara/blob/main/docs/pdf/UTS_2415051068_I_Kadek_Dwi_Bajaskara.pdf
Git Log	Lihat bagian "Bukti Proses Pengembangan (Git Log)"
Catatan kendala utama	Rate limit AniList API (90 request/menit kondisi normal; 30 request/menit saat kondisi degraded) diatasi dengan caching 2 jam dan penanganan HTTP 429; mode offline diimplementasikan untuk mengantisipasi koneksi terputus.
Lampiran	https://drive.google.com/drive/folders/1QtaqE8D0066MRBD4mLoqBwQQGTTSvgea?usp=sharing